

ArchVision 3D AI

Documentación del proyecto

1. Últimos cambios · 2. Guía de uso · 3. Guía de desarrollo · 4. Problemas conocidos

Rama `Deve1op` · commit `a395906` · 27 de septiembre de 2026

1. Últimos cambios

Este documento resume la historia reciente de la rama `Deve1op` , del más nuevo al más antiguo.

Resumen de ramas

Rama	Último commit	Estado frente a <code>main</code>
<code>Deve1op</code>	<code>a395906</code> Fix: poner suelos y puertas, aparte se agrega opcion para mover la vista 3D	1 commit por delante: tiene cambios que aún no están en <code>main</code>
<code>main</code>	<code>d3f2340</code> Merge pull request #4 from Txllagts/Pre-release	Estable
<code>Pre-release</code>	Merge pull request #3 from Txllagts/Backend-Ia	Integrada en <code>main</code>
<code>Backend-Ia</code>	<code>45beb28</code> feat: seleccion multiple, mover y rotar objetos en 2D/3D	Integrada en <code>main</code> y <code>Deve1op</code>
<code>Frontend-y-ui</code>	<code>f8c62ba</code> feat: plataforma ArchVision 3D AI (fases 1-7)	Sin cambios desde agosto

Historial

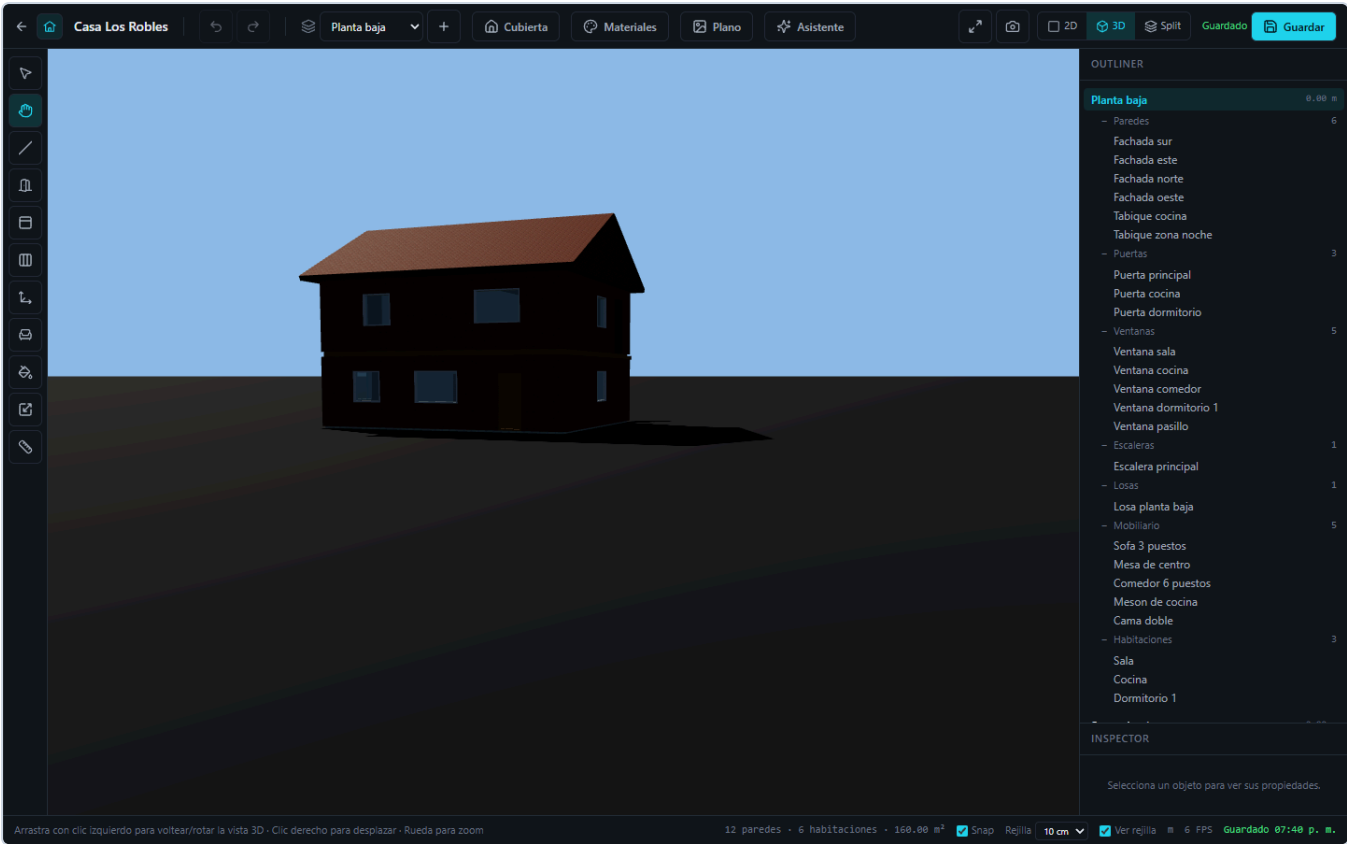
Fecha	Commit	Autor	Descripción
2026-09-27	<code>a395906</code>	Jasbeith Prada	Suelos de habitación con material, herramienta Mover y rotar vista 3D (<code>H</code>), eliminación de la página <i>Equipo</i>
2026-09-27	<code>66455d2</code>	Jasbeith Prada	Merge de <code>Backend-Ia</code> en <code>Deve1op</code>
2026-09-27	<code>45beb28</code>	Snayder07	Selección múltiple, mover y girar objetos en 2D/3D
2026-09-21	<code>0d25dc2</code>	Jasbeith Prada	fix(storage): índice posiblemente indefinido en <code>readUInt32BE</code>
2026-09-21	<code>29bb443</code>	Jasbeith Prada	fix(ci): módulos de almacenamiento que faltaban y <code>.gitignore</code>
2026-09-21	<code>b080abf</code>	Jasbeith Prada	README más claro y correcciones de CI
2026-09-21	<code>42fc0c9</code>	Txllagts	Importación de plano, conexión a Supabase y página <i>Equipo</i>
2026-08-13	<code>f8c62ba</code>	JuanDavid-dev-lang	Plataforma inicial (fases 1 a 7)

`a395906` : suelos, puertas y vista 3D libre

Rama: `Deve1op` · **Autor:** Jasbeith Prada · **11 archivos, +195 / -885 líneas**

Nueva herramienta: Mover y rotar vista 3D (`H`)

Hay un botón nuevo con icono de mano en la barra de herramientas izquierda, justo debajo de *Seleccionar*. También se activa con las teclas **H** o **Q** y con la paleta de comandos (**Ctrl+K**).



Acción	Con la herramienta <i>Mover vista</i> (H)	Con otras herramientas
Clic izquierdo + arrastrar	Orbita (rota) la cámara	Selecciona o dibuja
Clic derecho + arrastrar	Desplaza (pan) la cámara	Orbita la cámara
Botón central + arrastrar	Desplaza la cámara	Desplaza la cámara
Rueda	Zoom	Zoom

Con esta herramienta la cámara puede bajar **por debajo del plano del suelo** ($\text{maxPolarAngle} = \pi - 0.05$). Así se puede mirar la casa desde abajo. Con las demás herramientas el límite sigue en el horizonte. En la vista 2D, arrastrar con la herramienta activa desplaza el plano y el cursor cambia a una mano.

Mientras está activa, los clics sobre objetos no seleccionan ni resaltan nada. Así el usuario no mueve un objeto por accidente al navegar.

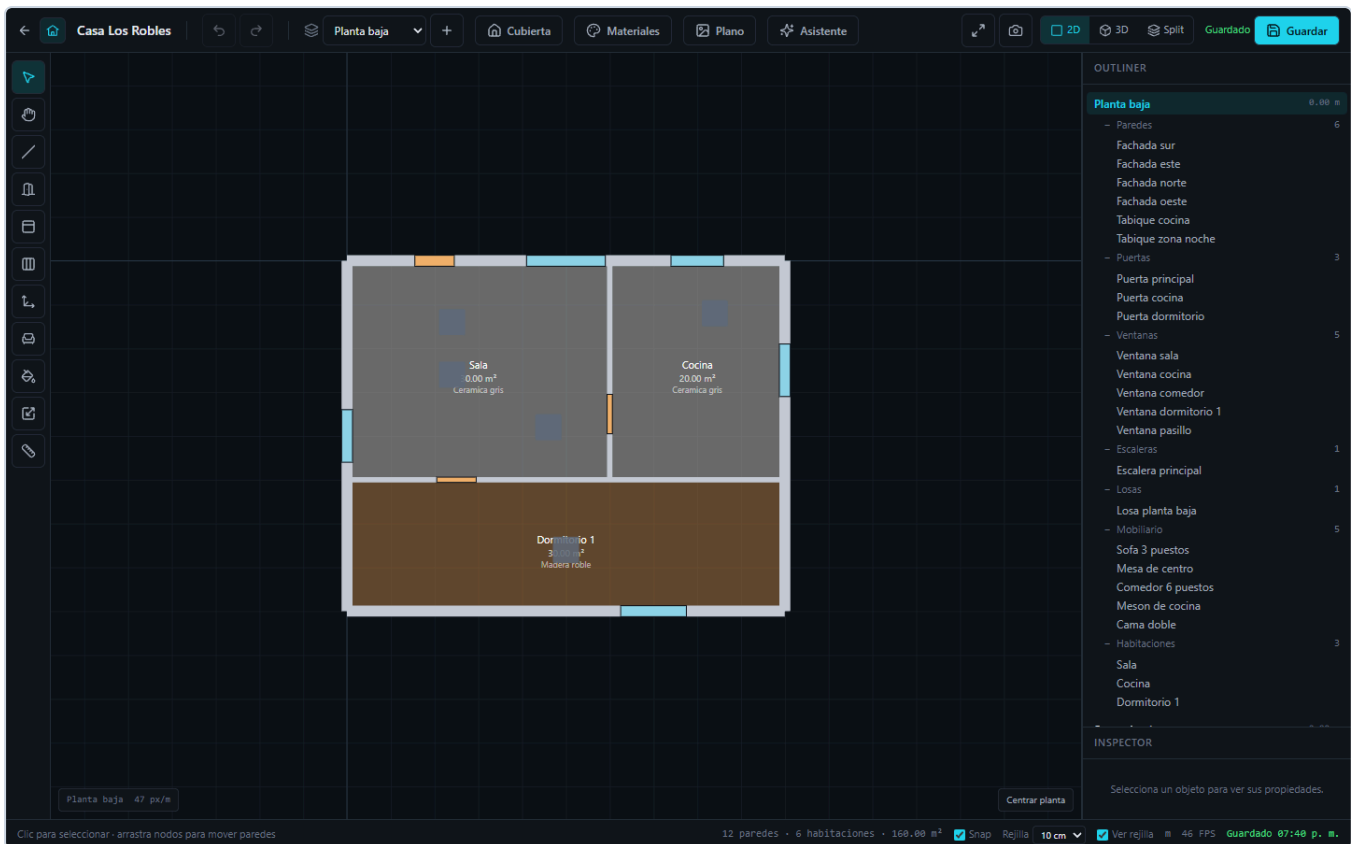
Archivos: `toolbar.tsx` , `use-shortcuts.ts` , `command-palette.tsx` , `status-bar.tsx` , `store.ts` (nuevo `ToolId "pan"`) , `viewport-3d.tsx` , `viewport-2d.tsx` .

Suelos de habitación con material

Cada habitación detectada tiene ahora una malla de suelo propia (`RoomFloorObject` en `scene-objects.tsx`). Es una capa de 2 mm sobre el nivel de la planta y puede recibir materiales con el pincel (**G**) o arrastrando una muestra.

- **En 3D:** sin material, el suelo es invisible pero sigue ahí para que el raycaster lo detecte. Al pasar el ratón por encima se resalta en azul.

- **En 2D:** si la habitación tiene material, el polígono se rellena con el color base del material al 55 % de opacidad. Debajo del área aparece el nombre del material.



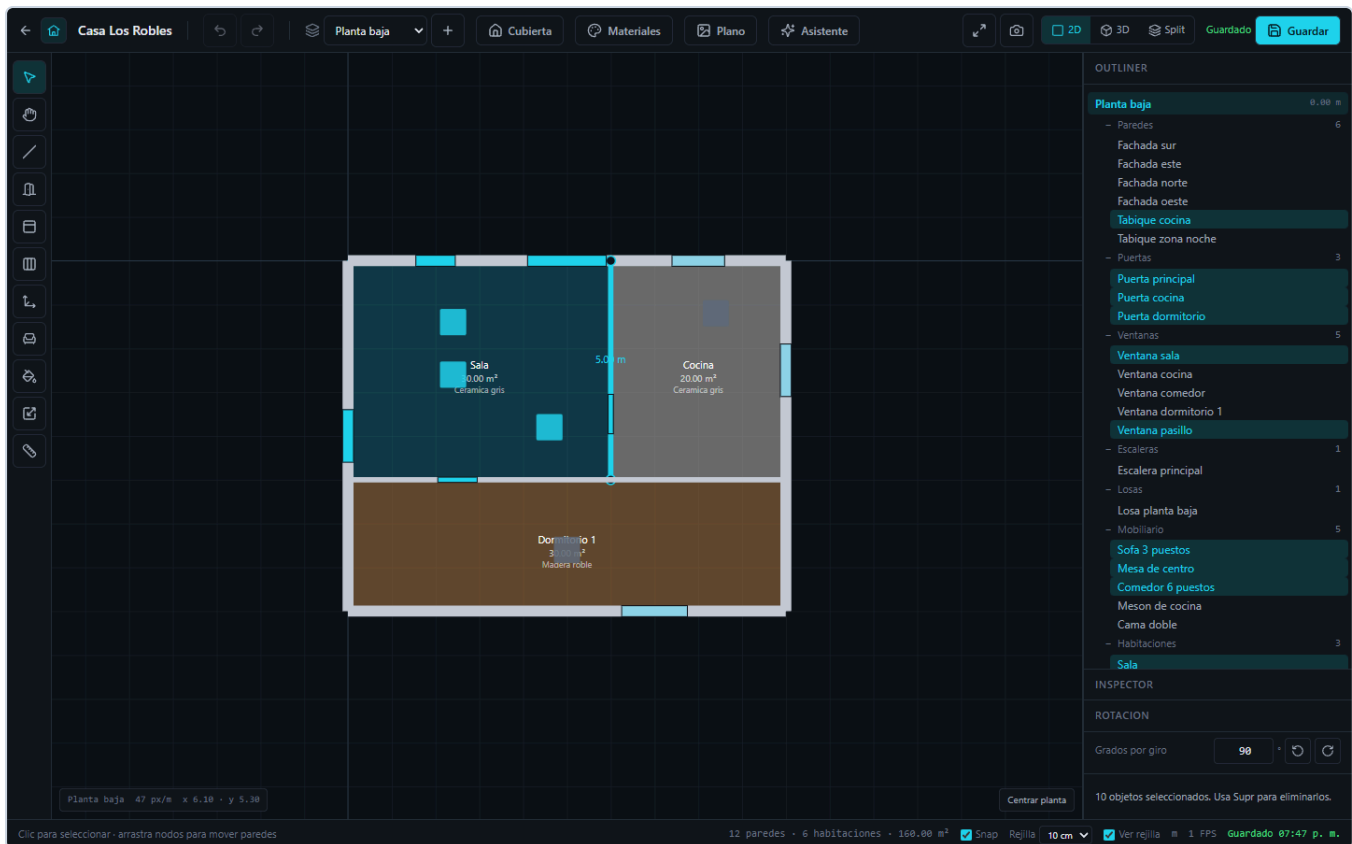
Página *Equipo* eliminada

Se borraron `apps/web/src/app/(app)/team/page.tsx` y `components/team/role-assignment-view.tsx` (852 líneas), además de su enlace en la barra lateral. La página se había agregado en `42fc0c9`.

45beb28 : selección múltiple, mover y girar

Rama: `Backend-Ia` · **Autor:** `Snayder07` · **17 archivos, +1888 / –135 líneas**

- **Selección por recuadro (marquee)** en 2D y 3D:
 - De izquierda a derecha: selecciona los objetos que quedan **completamente dentro**.
 - De derecha a izquierda: selecciona los que **tocan** el recuadro.
- `Ctrl+A` selecciona todo lo visible de la planta activa.
- **Arrastrar objetos en 3D:** si el objeto ya forma parte de una selección múltiple, se mueve todo el grupo. Al soltar se emite un único comando `TRANSFORM_OBJECTS`, que se deshace con un solo `Ctrl+Z`.
- **Giro interactivo:** un tirador circular sobre el pivote de la selección. Por defecto el giro es libre; con `Shift` se ajusta al incremento guardado en el inspector (de 1° a 360°, se guarda en `localStorage`).
- **Cubiertas con pose propia:** `Roof` tiene ahora `position` y `rotationY` opcionales. Antes, girar una cubierta la deformaba. Las escenas antiguas se siguen leyendo, con giro 0.
- Nuevo módulo `packages/shared/src/selectable-bounds.ts`: es la fuente única de las cajas envolventes de cada entidad y de sus dependencias (un vano depende de su muro; una habitación, de sus muros).
- Nuevas pruebas en `editor.test.ts` y `selectable-bounds.test.ts`.



Detalle técnico: [docs/SELECTION_AND_TRANSFORMS.md](#).

42fc0c9 → 0d25dc2 : Supabase, almacenamiento y CI

- **Supabase:** el esquema de Prisma usa ahora `provider = "postgresql"` con `url` y `directUrl`. `.env.example` explica `DATABASE_URL` (Transaction Pooler, puerto 6543) y `DIRECT_URL` (Session Pooler, puerto 5432).
- **Almacenamiento:** se agregaron `apps/web/src/lib/storage/index.ts` y `sniff.ts`, que faltaban en el repositorio y rompían el typecheck de CI. `sniff.ts` detecta el tipo real de un archivo por sus primeros bytes, no por la extensión.
- **README:** se reescribió con una guía de puesta en marcha para los miembros del equipo.

Impacto para el equipo

1. Hay que llevar `Develop` a `main`. La herramienta de vista 3D y los suelos todavía no están en `main`.
2. Las migraciones de Prisma no coinciden con Supabase. Ver [Problemas conocidos](#).
3. Una prueba de `three-engine` falla después de los cambios en la casa demo. Ver [Problemas conocidos](#).

2. Guía de uso

Recorrido por ArchVision 3D AI desde el registro hasta el modelado en el editor. Las capturas corresponden a la cuenta de demostración y al proyecto **Casa Los Robles**.

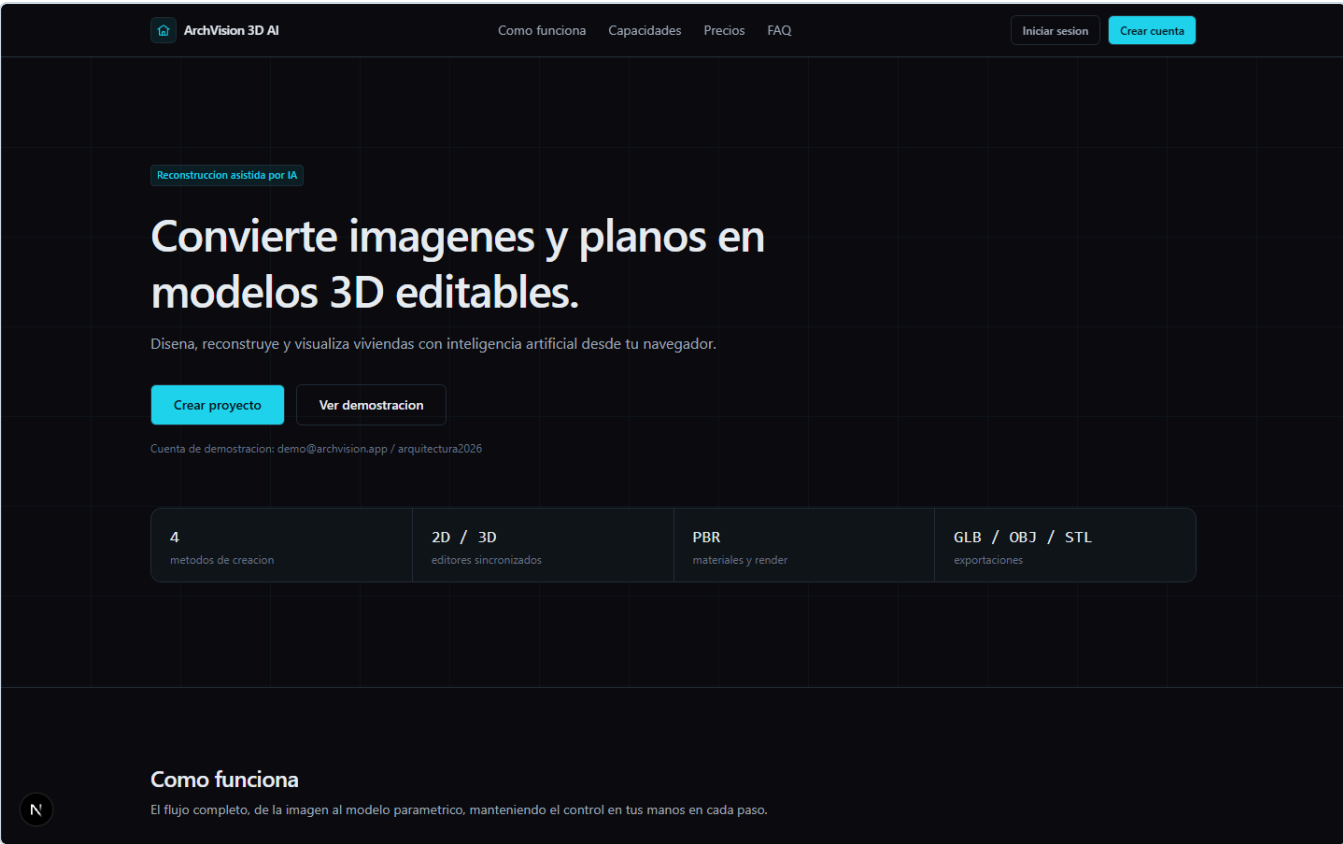
Cuenta de demostración (después de ejecutar `pnpm db:seed`): `demo@archvision.app` / `arquitectura2026`

Contenido

1. Página de inicio
2. Registro e inicio de sesión
3. Panel principal
4. Proyectos
5. El editor
6. Vistas 2D, 3D y dividida
7. Navegar por la vista 3D
8. Herramientas de modelado
9. Seleccionar, mover y girar
10. Materiales
11. Cubiertas
12. Importar un plano
13. Asistente, revisión y tutorial
14. Paleta de comandos
15. Configuración, facturación y papelera
16. Atajos de teclado

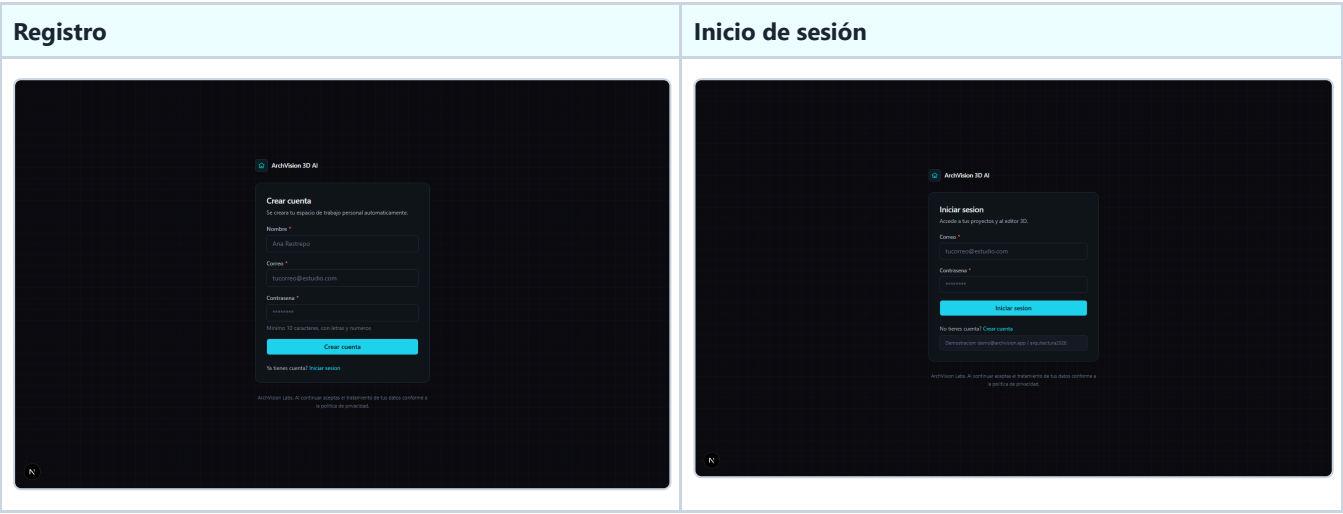
2.1 Página de inicio

Al abrir `http://localhost:3000` aparece la página pública del producto. Desde aquí se accede al registro y al inicio de sesión.



2.2 Registro e inicio de sesión

- **Crear cuenta** (`/register`): pide nombre, correo y contraseña.
- **Iniciar sesión** (`/login`): pide correo y contraseña. La sesión dura 30 días, o el valor de `SESSION_TTL_DAYS` .

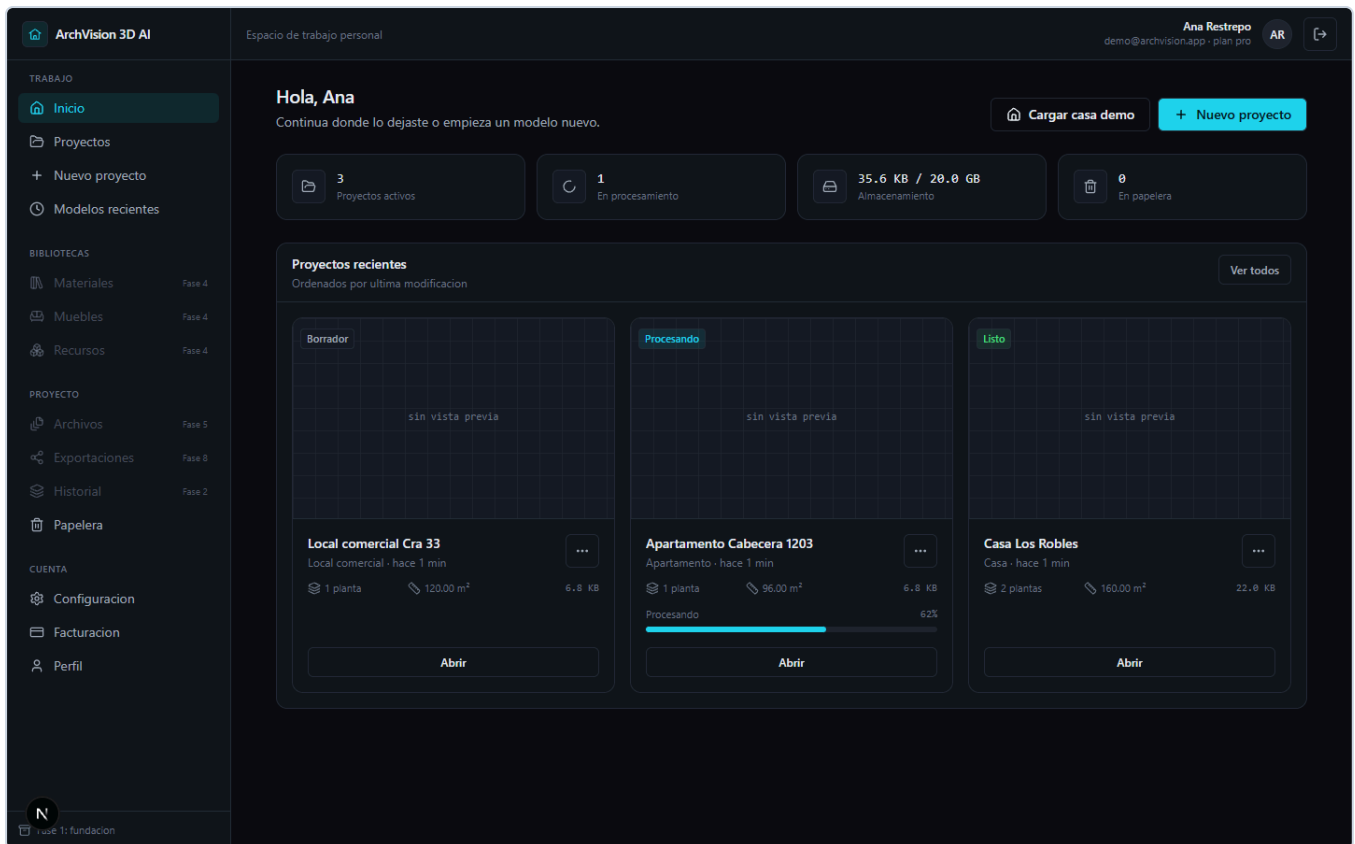


2.3 Panel principal

Después de iniciar sesión se abre **Inicio** (`/dashboard`). Muestra:

- **Tarjetas de resumen:** proyectos activos, en procesamiento, almacenamiento usado y proyectos en la papelera.
- **Proyectos recientes**, ordenados por última modificación.

- **Cargar casa demo:** crea una copia de la casa de ejemplo de dos plantas.
- **Nuevo proyecto.**



La barra lateral agrupa las secciones. Las que aún no existen aparecen en gris con la fase en la que llegarán (por ejemplo *Exportaciones · Fase 8*).

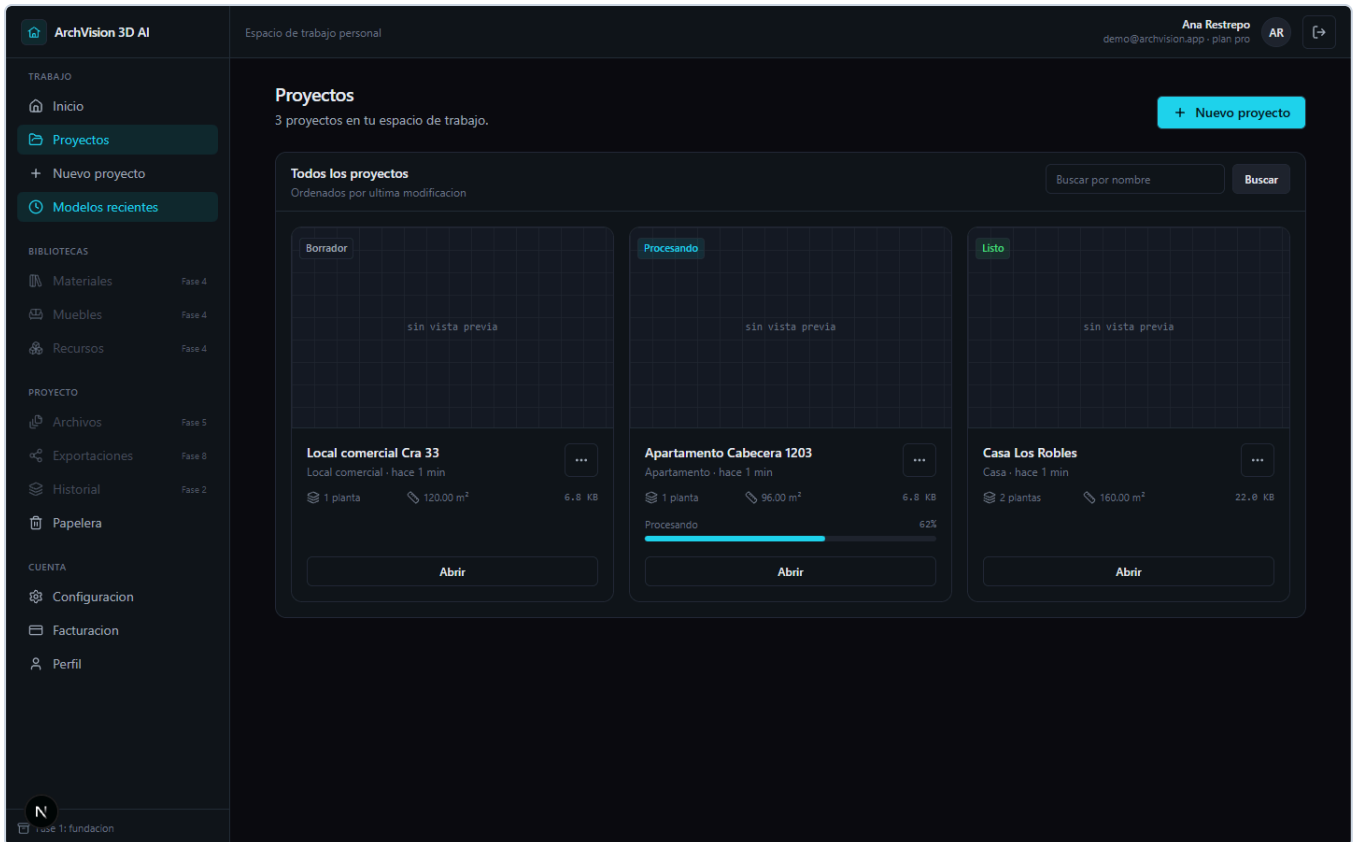
2.4 Proyectos

Lista de proyectos

Proyectos (`/projects`) lista todos los proyectos del espacio de trabajo y tiene un buscador por nombre. Cada tarjeta muestra:

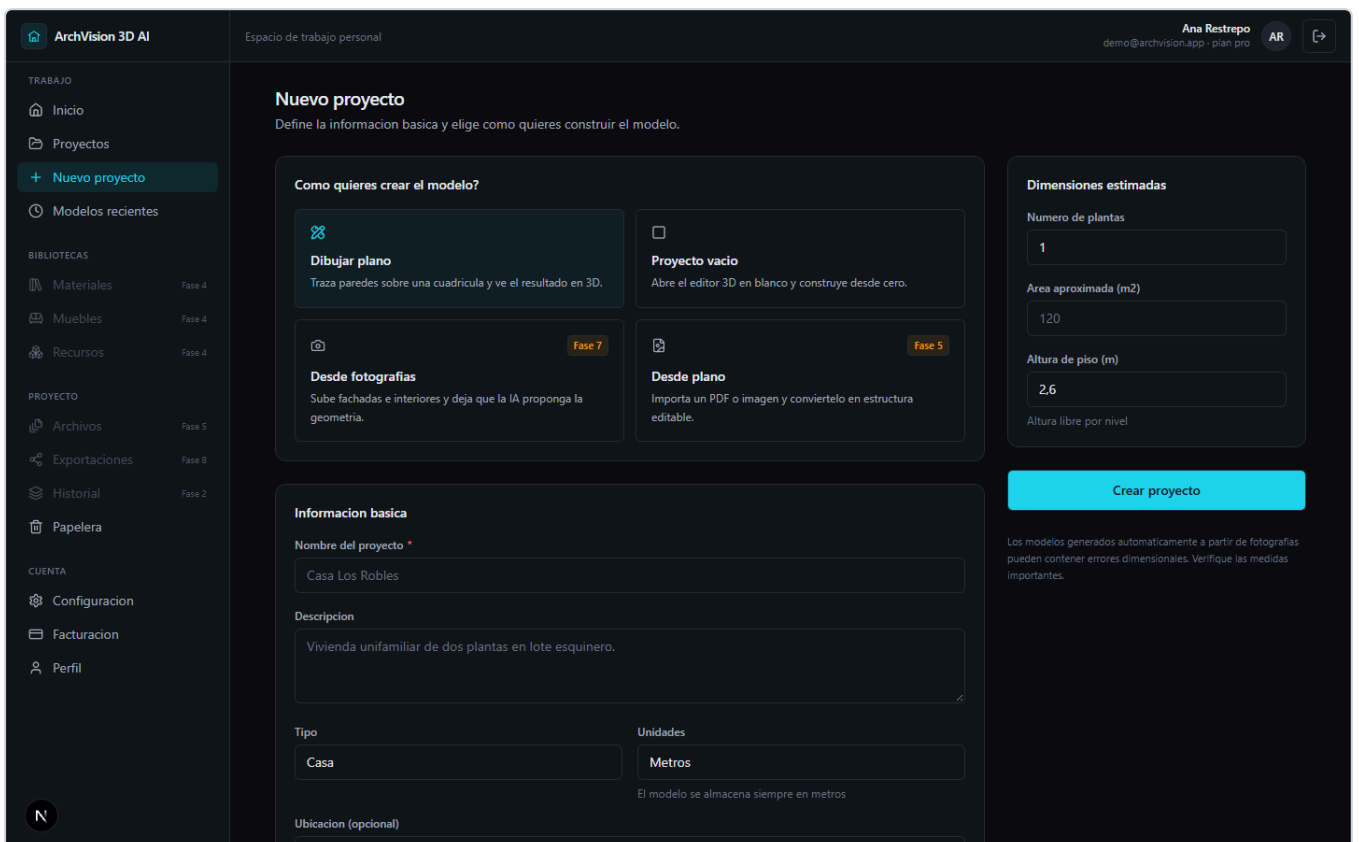
- Estado: *Borrador*, *Procesando* o *Listo*.
- Número de plantas, área y tamaño.
- Botón **Abrir**.

El menú `...` de cada tarjeta permite **duplicar**, **ver versiones** o **enviar a la papelera**.



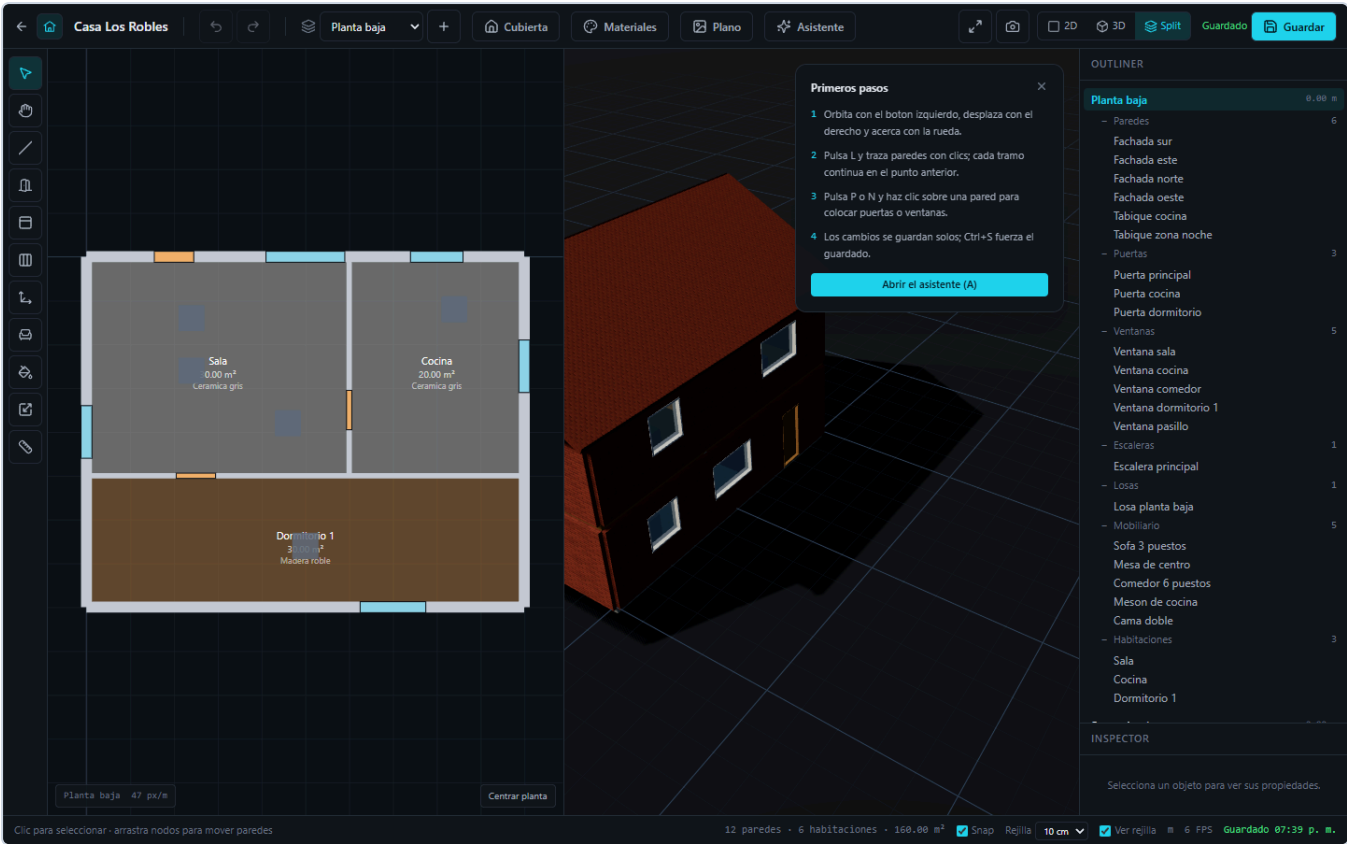
Crear un proyecto

Nuevo proyecto (`/projects/new`) pide nombre, tipo de edificación y datos básicos. Al guardar se crea una escena vacía y se abre el proyecto.



2.5 El editor

Al abrir un proyecto y pulsar **Abrir editor** se llega a `/projects/{id}/editor` . La primera vez aparece la tarjeta **Primeros pasos** con los controles básicos.



Partes del editor:

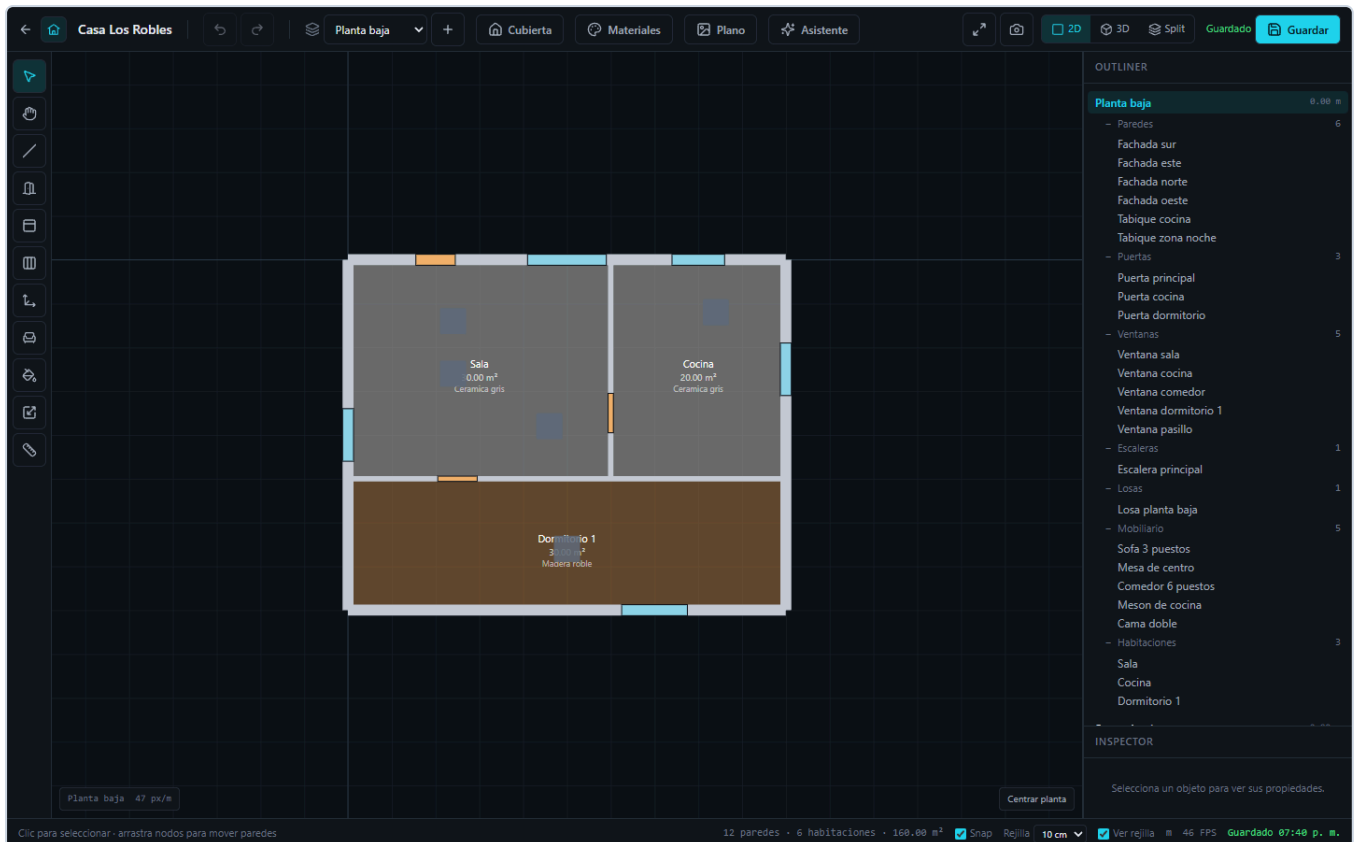
Zona	Contenido
Barra superior	Volver, nombre del proyecto, deshacer/rehacer, selector de planta y + para añadir planta, botones Cubierta , Materiales , Plano , Asistente , pantalla completa, captura, modo de vista (2D / 3D / Split), estado de guardado y Guardar
Barra de herramientas (izquierda)	Seleccionar, mover vista, pared, puerta, ventana, columna, escalera, mobiliario, pincel, calibrar y medir
Visor (centro)	Plano 2D, modelo 3D o ambos
Outliner (derecha, arriba)	Árbol de la escena por planta: paredes, puertas, ventanas, escaleras, losas, cubiertas, mobiliario y habitaciones. Cada elemento se puede ocultar o bloquear
Inspector (derecha, abajo)	Propiedades del objeto seleccionado
Barra de estado (abajo)	Ayuda de la herramienta activa, estadísticas del modelo, <i>Snap</i> , tamaño de rejilla, <i>Ver rejilla</i> , FPS y hora del último guardado

Todo se guarda solo. El indicador *Guardado* de la barra superior confirma el último guardado. **Ctrl+S** guarda al momento.

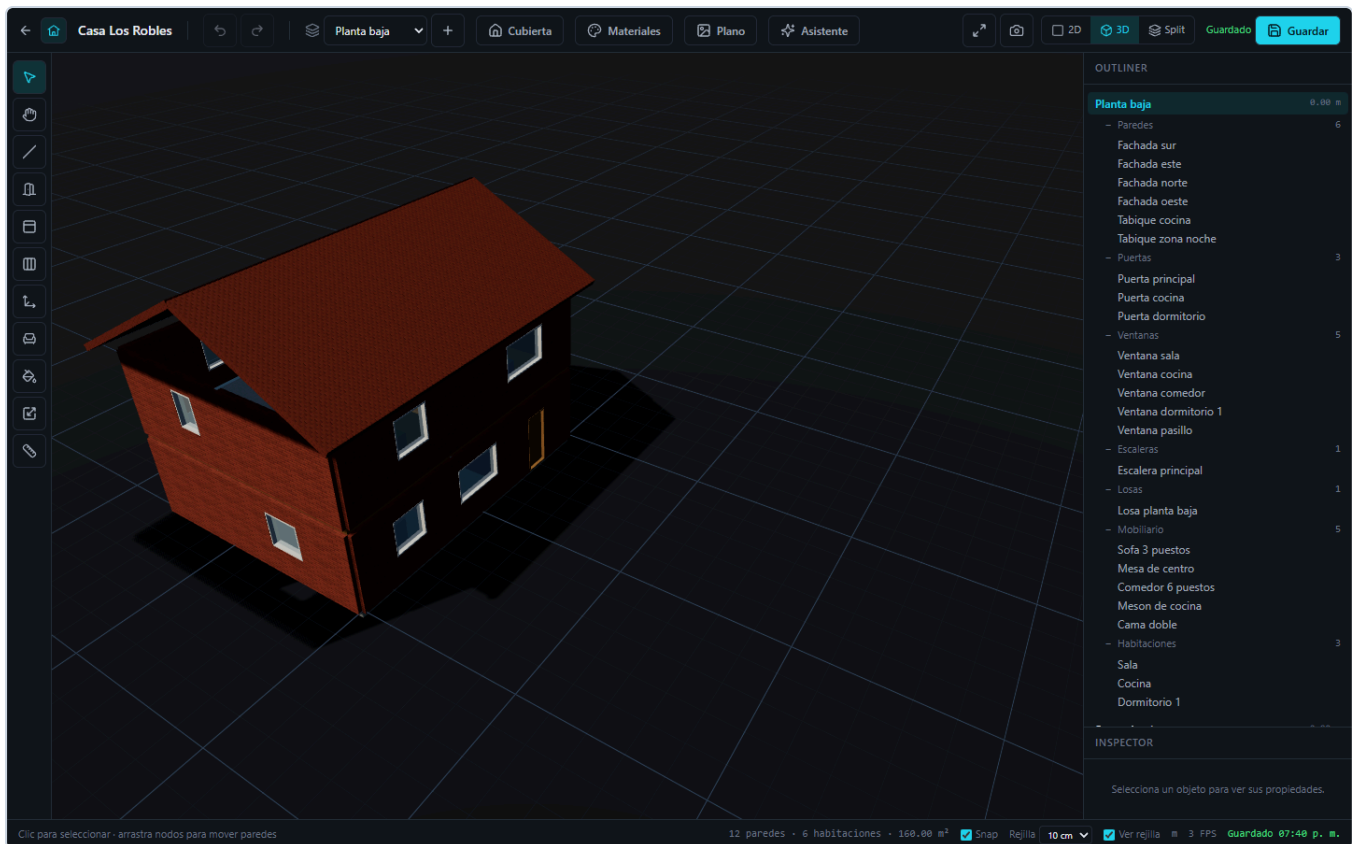
2.6 Vistas 2D, 3D y dividida

Los botones **2D**, **3D** y **Split** de la barra superior cambian el modo de vista. La tecla **Tab** alterna entre ellos. Las dos vistas editan el mismo modelo: un cambio en la planta 2D aparece al instante en el 3D, y al revés.

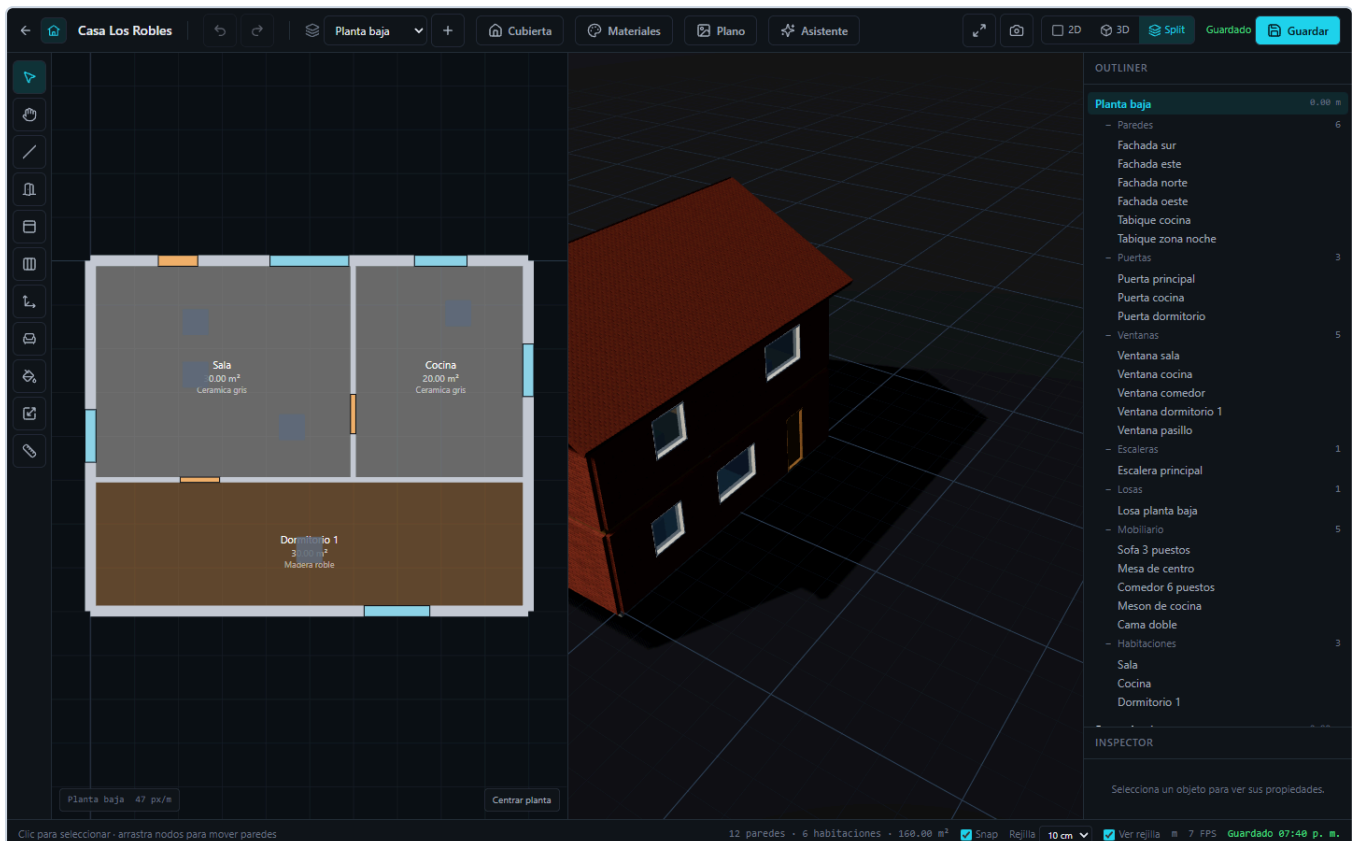
Vista 2D (planta): muestra muros, vanos (puertas en naranja, ventanas en celeste), habitaciones con su nombre, área y material de suelo, y el mobiliario.



Vista 3D: muestra el modelo con materiales, sombras y cubierta.



Vista dividida (Split): muestra el plano a la izquierda y el modelo 3D a la derecha.



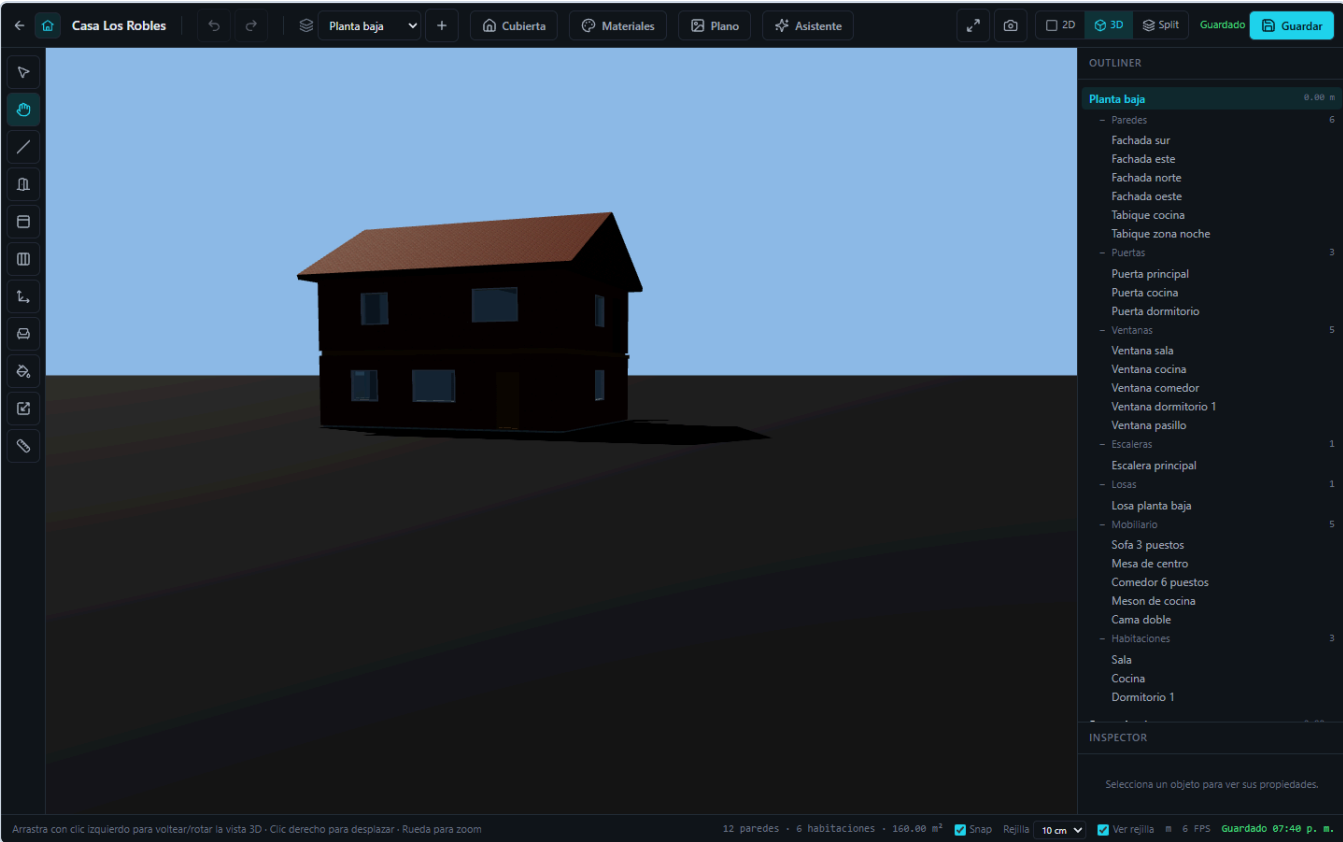
Encuadres rápidos en 3D: **1** vista frontal, **2** lateral, **3** superior, **0** perspectiva y **F** centra la selección. En 2D, el botón **Centrar planta** reencuadra el plano.

2.7 Navegar por la vista 3D

Nuevo en Develop : herramienta **Mover y rotar vista 3D** (icono de mano, tecla **H**).

Acción	Herramienta <i>Mover vista</i> (H)	Herramienta <i>Seleccionar</i> (V)
Clic izquierdo + arrastrar	Rota la cámara alrededor del modelo	Selecciona o arrastra objetos
Clic derecho + arrastrar	Desplaza la cámara	Rota la cámara
Botón central + arrastrar	Desplaza la cámara	Desplaza la cámara
Rueda del ratón	Zoom	Zoom

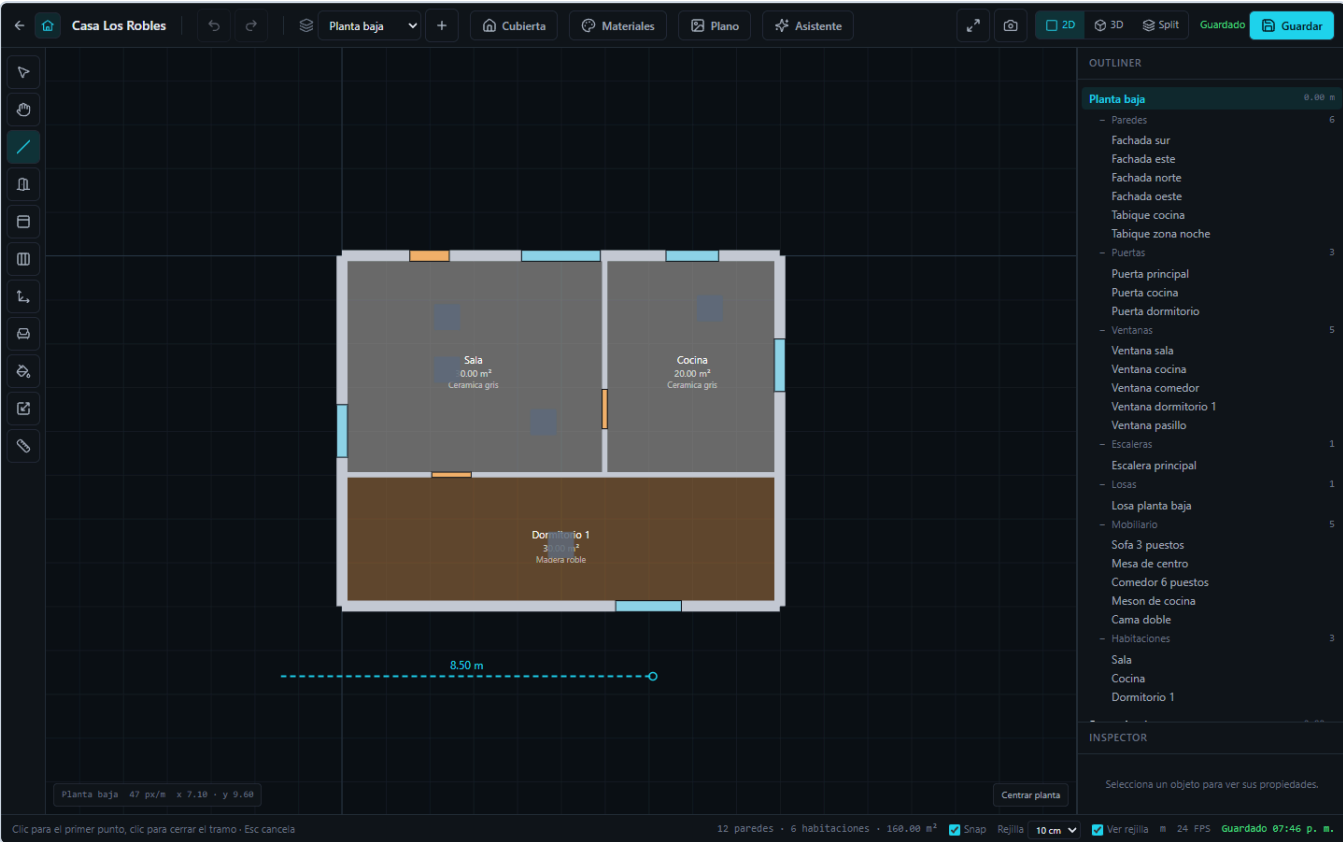
Con **Mover vista** la cámara también puede ir por debajo del suelo. Mientras está activa no se seleccionan objetos, así no se mueve nada por accidente.



2.8 Herramientas de modelado

Herramienta	Tecla	Cómo se usa
Seleccionar	V	Clic para seleccionar; arrastrar nodos de pared para moverlos
Mover vista	H / Q	Ver 2.7
Pared	L	Clic para el primer punto y clic para cada tramo siguiente. Los tramos se encadenan. Esc termina
Puerta	P	Clic sobre una pared. El hueco se abre al instante
Ventana	N	Clic sobre una pared
Columna	C	Clic en el plano
Escalera	botón	Escalera paramétrica
Mobiliario	B	Coloca muebles del catálogo
Pintar material	G	Abre la biblioteca y carga el pincel. Ver 2.10
Calibrar plano	K	Fija la escala de un plano importado
Medir	M	Clic en dos puntos para ver la distancia

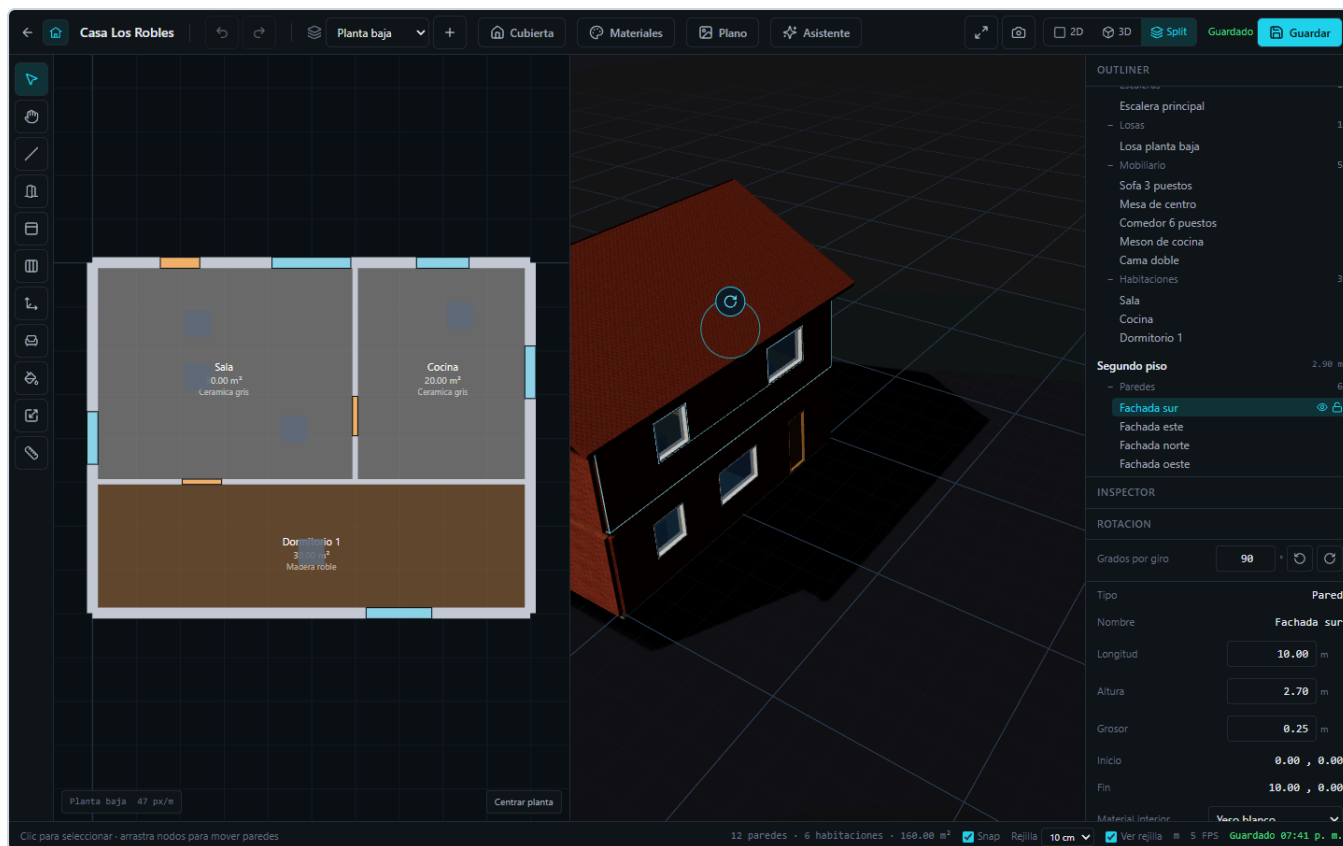
Dibujar una pared: la línea discontinua muestra la longitud en tiempo real. El *snapping* se ajusta a vértices, puntos medios, ejes ortogonales y rejilla. Se desactiva con la casilla *Snap* de la barra de estado.



Al cerrar un recinto con paredes, la **habitación se detecta sola** y aparece con su área en la planta y en el outliner.

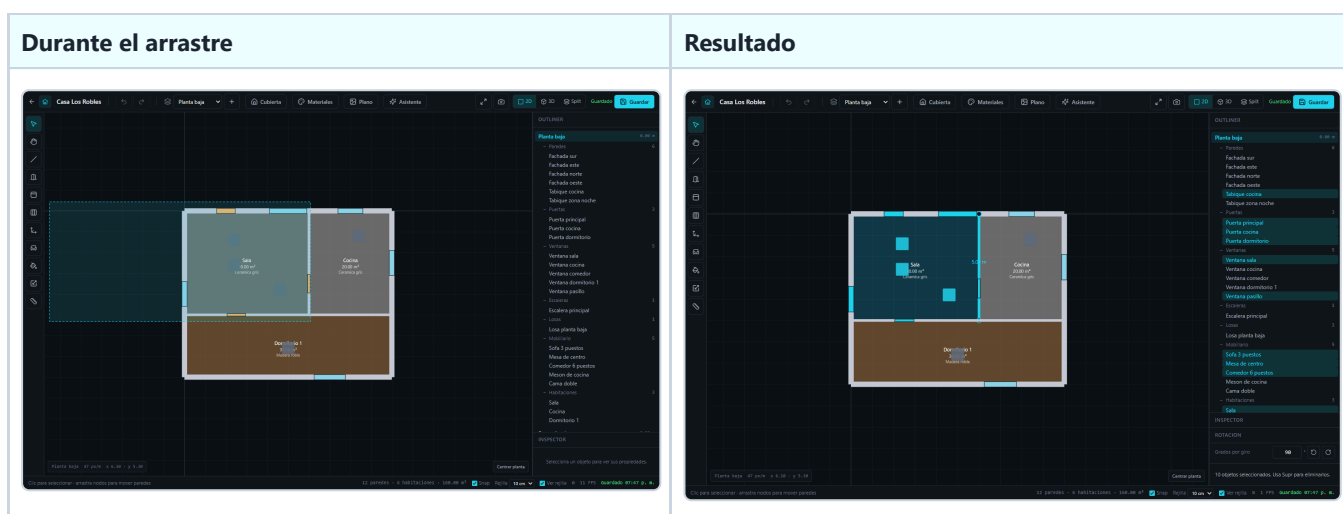
2.9 Seleccionar, mover y girar

Selección simple: clic sobre un objeto en el visor o en el outliner. El inspector muestra sus propiedades editables. En una pared: longitud, altura, grosor, coordenadas de inicio y fin, y material por cara.



Selección múltiple:

- **Shift + clic** añade objetos a la selección.
- **Recuadro:** arrastrar en una zona vacía.
 - **De izquierda a derecha:** selecciona lo que queda **completamente dentro**.
 - **De derecha a izquierda:** selecciona lo que **toca** el recuadro.
- **Ctrl+A** selecciona todo lo visible en la planta activa.



Mover (3D): arrastra un objeto seleccionado. Si forma parte de un grupo, se mueve todo el grupo. Las puertas y ventanas arrastran su pared.

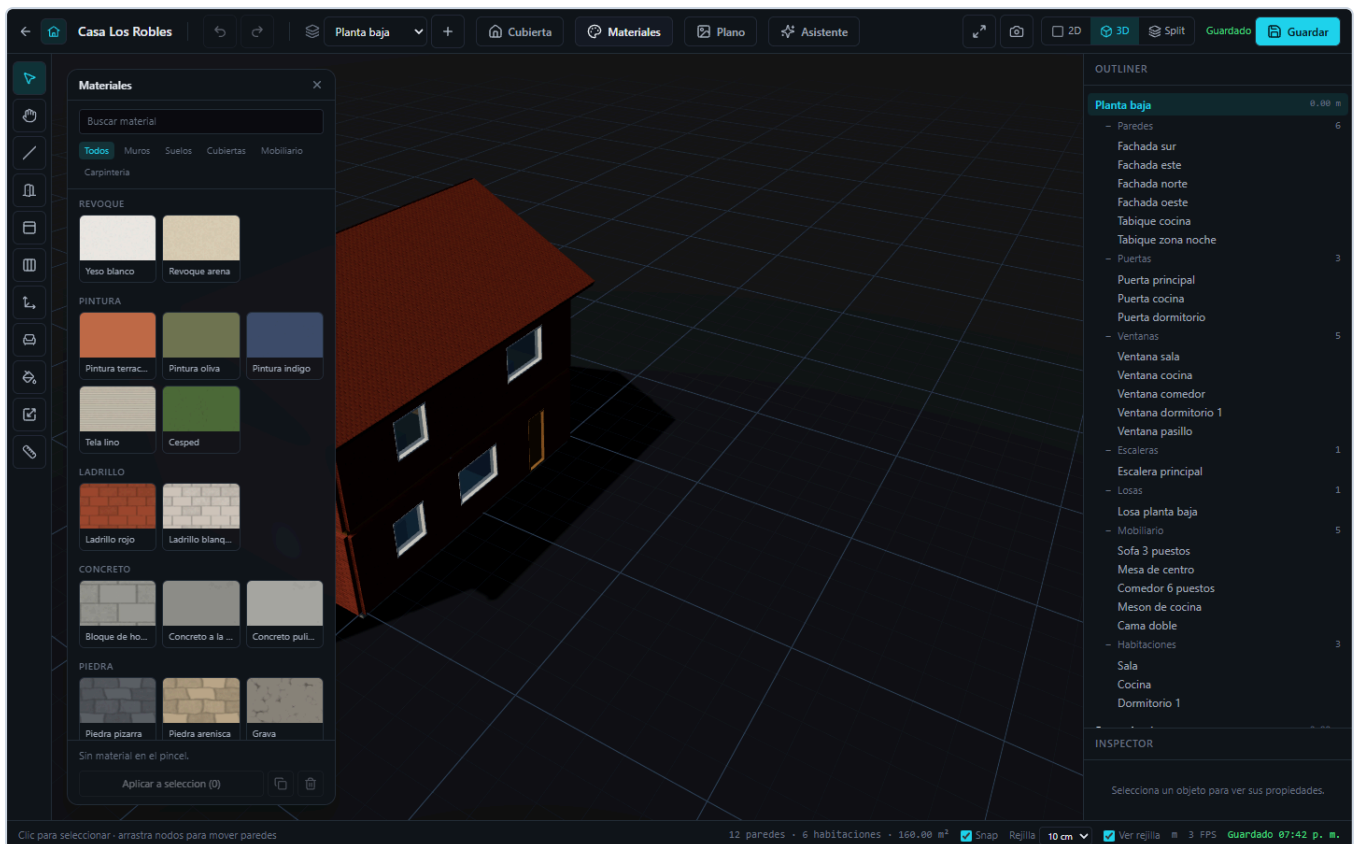
Girar (3D):

1. Selecciona uno o varios objetos.
2. Arrastra el **tirador circular** que aparece sobre la selección.
3. Para girar a saltos fijos, mantén **shift** mientras arrastras. El tamaño del salto se elige en el campo **Grados por giro** del inspector y se recuerda entre sesiones.
4. Los botones   del inspector giran exactamente ese número de grados.

Cada movimiento o giro se deshace con un solo **Ctrl+Z** . **Supr** elimina la selección.

2.10 Materiales

El botón **Materiales** (o la tecla **G**) abre la biblioteca de materiales. Tiene búsqueda y filtros por *Muros*, *Suelos*, *Cubiertas*, *Mobiliario* y *Carpintería*. Todos los materiales tienen textura procedural (ladrillo, madera, cerámica, piedra, concreto, pintura, etc.).



Hay tres formas de aplicar un material:

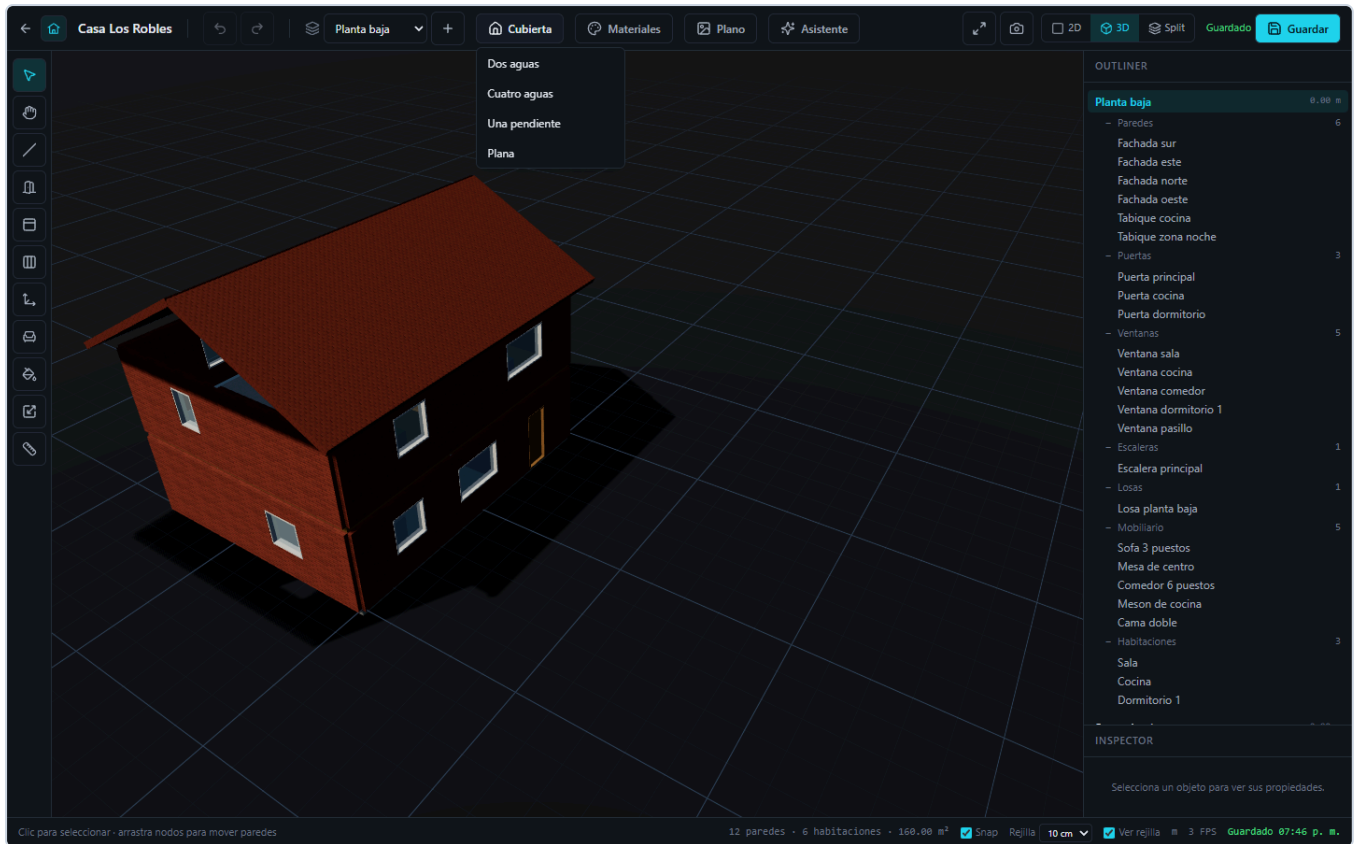
1. **Arrastrar** una muestra y soltarla sobre una superficie del modelo 3D.
2. **Pincel**: clic en una muestra para cargarla y después clic sobre las superficies, en 3D o en la planta.
3. **Aplicar a selección**: selecciona varios objetos y pulsa el botón inferior del panel.

Nuevo en DeveloP : los **suelos de las habitaciones** reciben material. En la planta 2D la habitación se colorea con el tono del material y muestra su nombre (en la captura, *Dormitorio 1* en madera de roble).

Los materiales del catálogo no se editan directamente. Primero se duplican con el icono de copiar.

2.11 Cubiertas

El botón **Cubierta** despliega los tipos disponibles: **dos aguas**, **cuatro aguas**, **una pendiente** y **plana**. La cubierta se genera sobre el contorno de la planta activa. Su pendiente, alero y espesor se ajustan en el inspector.

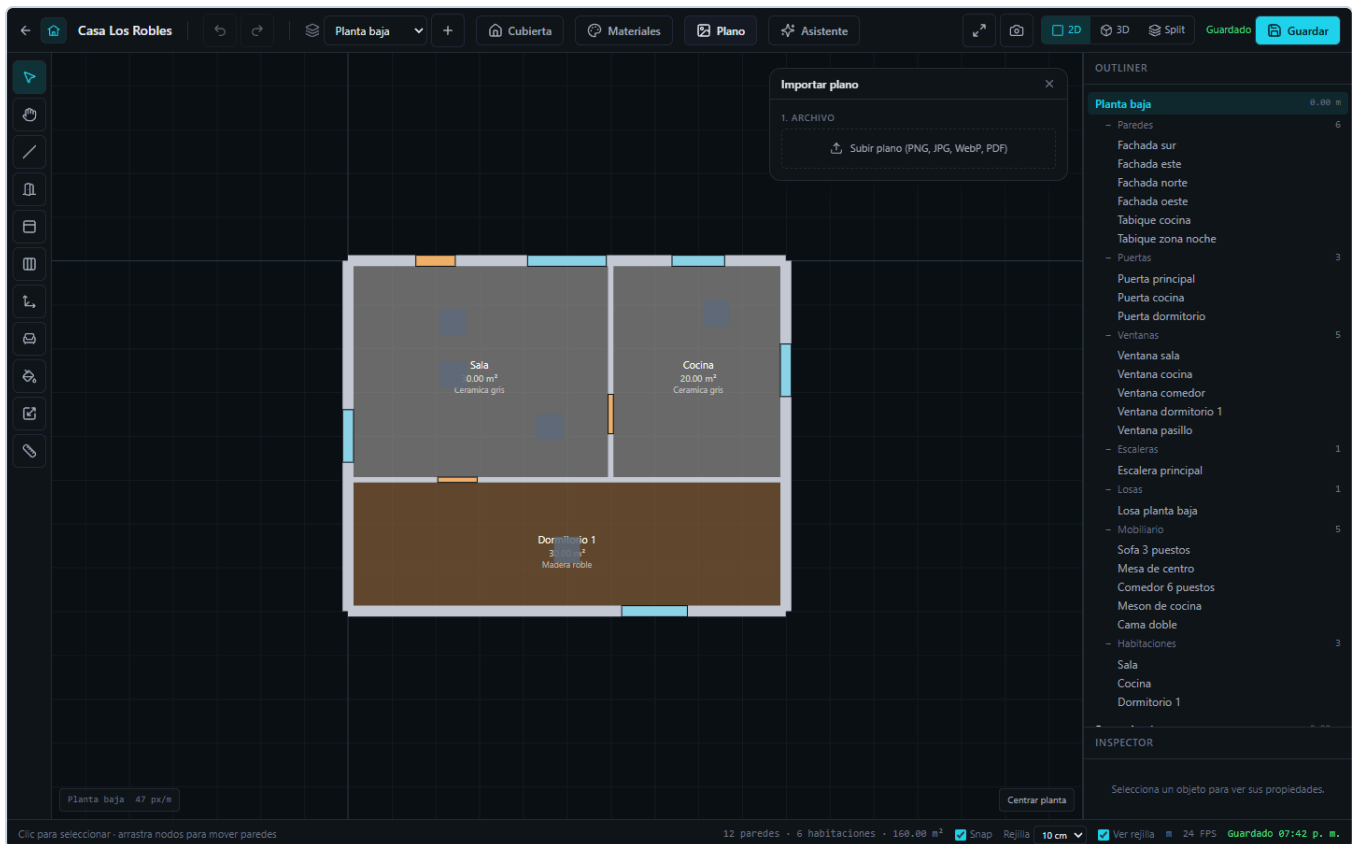


2.12 Importar un plano

El botón **Plano** abre el panel de importación:

1. **Subir** una imagen del plano (PNG, JPG, WebP o PDF).
2. **Calibrar** (K): marca sobre el plano dos puntos de una medida conocida y escribe su longitud real.
3. **Analizar**: la detección automática propone muros (umbral de Otsu y transformada de Hough).
4. **Revisar**: acepta solo los muros bien trazados. Todo se deshace con **Ctrl+Z**.

El plano queda como calco debajo del modelo para dibujar encima.

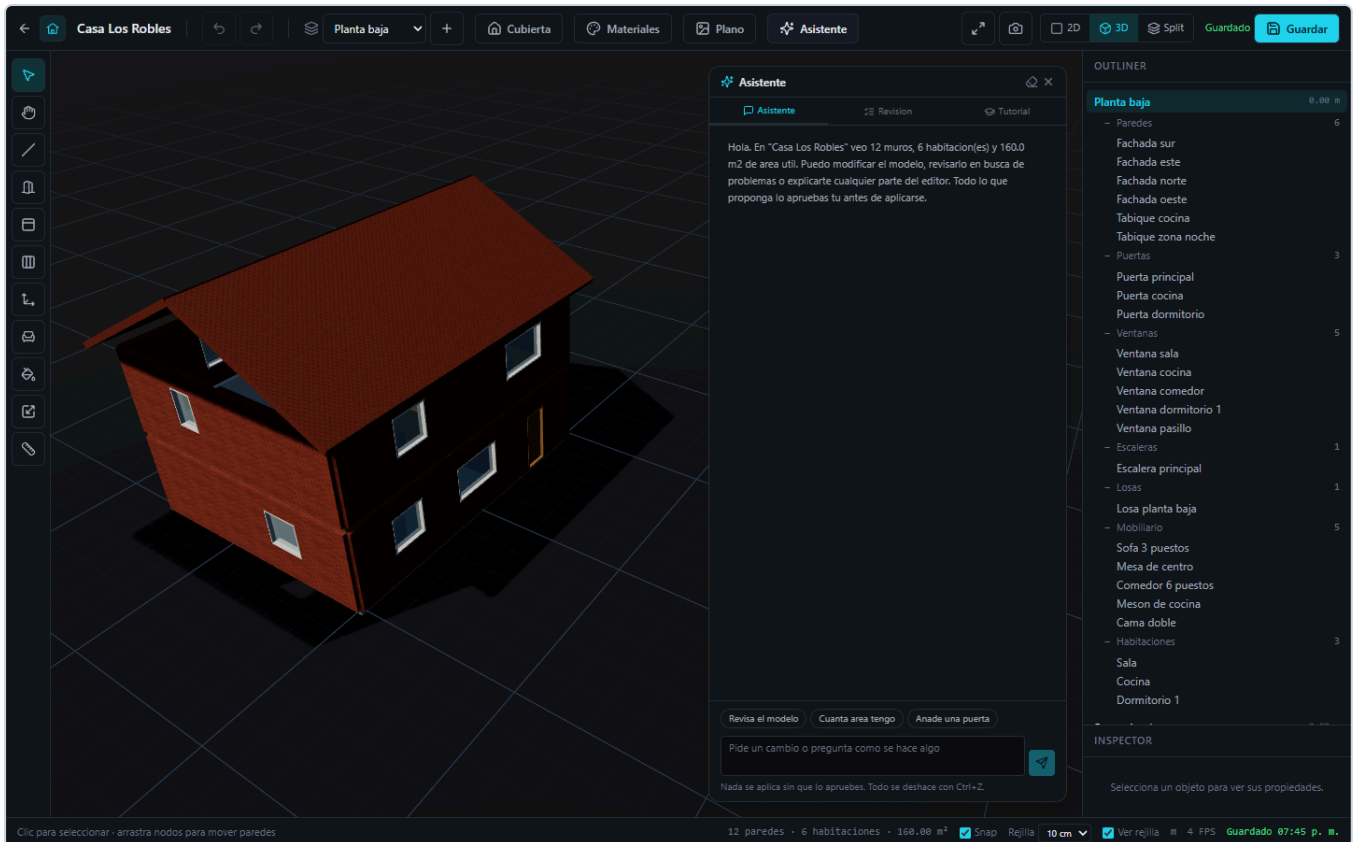


Detalles: [docs/PLAN_IMPORT.md](#).

2.13 Asistente, revisión y tutorial

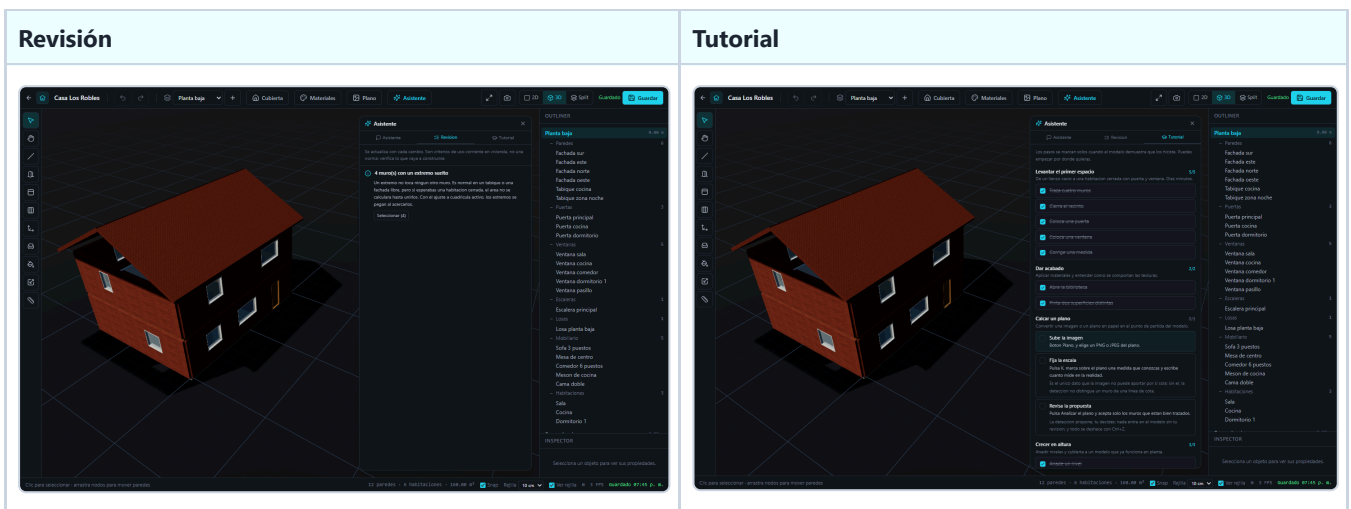
El botón **Asistente** (o la tecla **A**) abre un panel con tres pestañas.

Asistente: pide cambios en lenguaje natural, por ejemplo *"añade una puerta"* o *"¿cuánta área tengo?"*. El asistente propone los cambios y **nada se aplica sin tu aprobación**. Cada propuesta se deshace con **Ctrl+Z**.



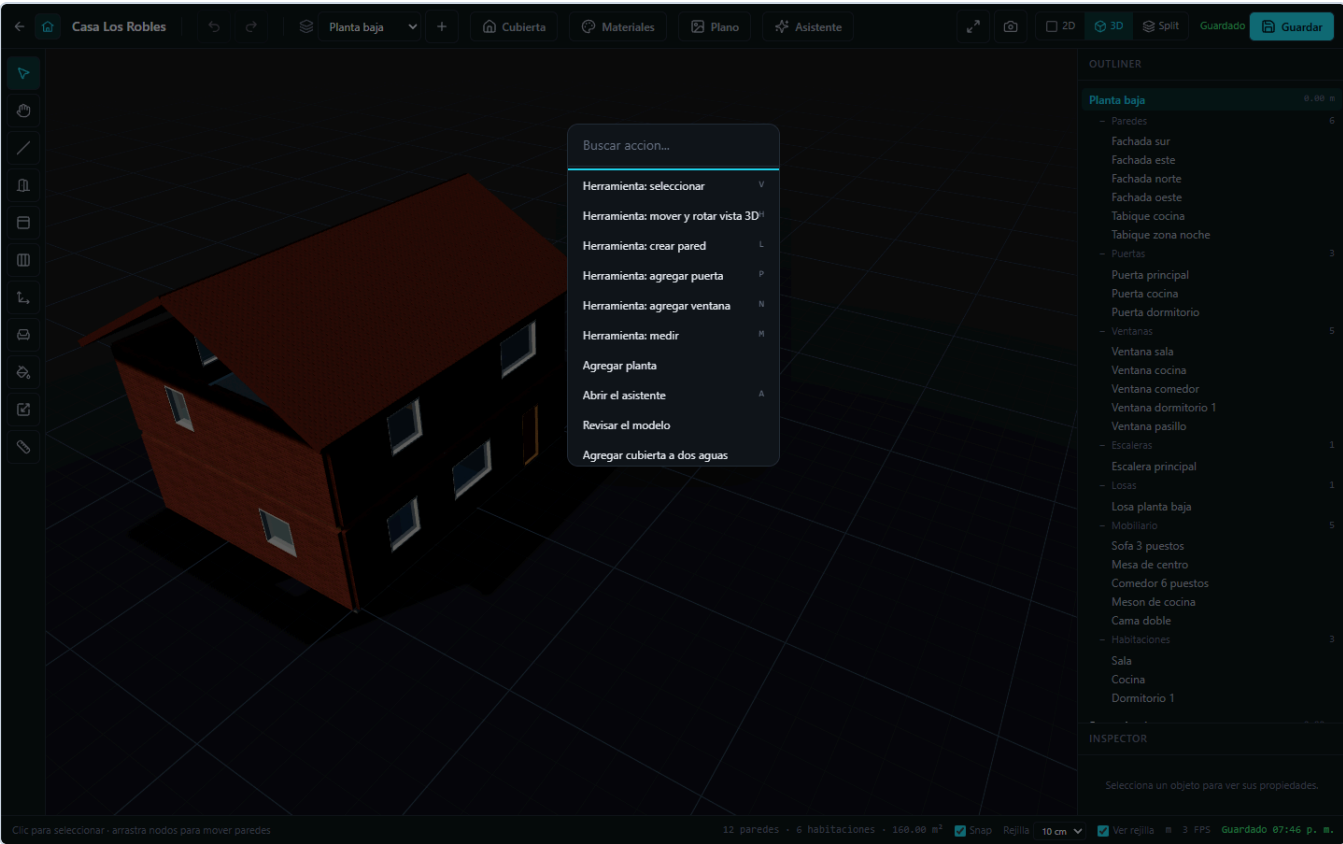
Revisión: analiza el modelo y avisa de problemas, por ejemplo muros sueltos, habitaciones sin acceso, vanos fuera del muro, muros duplicados, poca luz natural o escaleras incómodas. El botón **Seleccionar** lleva a los elementos afectados.

Tutorial: lista de pasos que se marcan solos a medida que trabajas.



2.14 Paleta de comandos

Ctrl+K abre la paleta de comandos. Escribe para filtrar y pulsa **Enter** para ejecutar la acción. Incluye todas las herramientas, añadir planta, abrir el asistente, revisar el modelo y añadir cubiertas.



2.15 Configuración, facturación y papelería

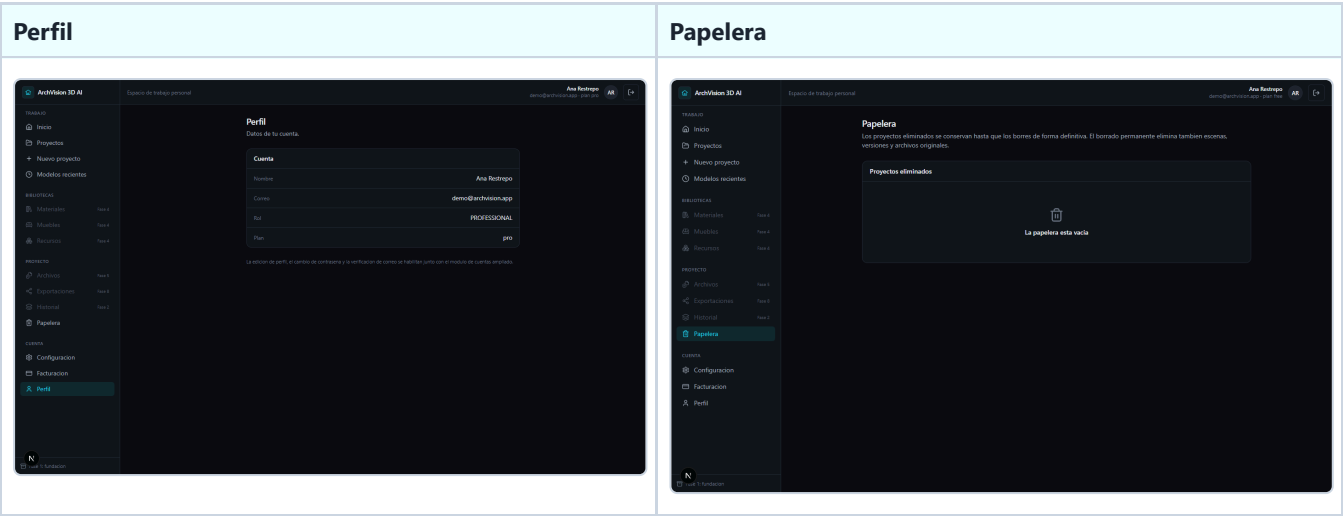
Sección	Ruta	Uso
Configuración	/settings	Preferencias de la cuenta y del espacio de trabajo
Perfil	/settings/profile	Nombre y datos del usuario
Facturación	/settings/billing	Plan actual, cambio a Pro o Studio (mensual o anual) y renovaciones
Papelera	/trash	Proyectos eliminados. Se pueden restaurar

Configuración

A screenshot of the ArchiVision 3D AI settings page. The left sidebar shows a navigation menu with options like 'Inicio', 'Proyectos', 'Modelos recientes', 'Archivos', 'Español', 'Papelera', 'Configuración', 'Facturación', and 'Perfil'. The main content area is titled 'Configuración' and shows settings for the user's account, including a 'Plan Free' section with details about the current plan and a 'Privacidad' section with information about data handling.

Facturación

A screenshot of the ArchiVision 3D AI billing page. The left sidebar is the same as the settings page. The main content area is titled 'Facturación' and shows a comparison of different subscription plans: 'Free', 'Pro', 'Studio', and 'Enterprise'. Each plan has a list of features and a 'Contratar' button. The 'Free' plan is highlighted as the current plan.



2.16 Atajos de teclado

Tecla	Acción
V	Seleccionar
H o Q	Mover y rotar vista 3D <i>(nuevo)</i>
L	Pared
P	Puerta
N	Ventana
C	Columna
B	Mobiliario
M	Medir
G	Pincel y biblioteca de materiales
K	Calibrar plano
A	Abrir o cerrar el asistente
F	Centrar la selección
1 / 2 / 3 / 0	Vista frontal / lateral / superior / perspectiva
Tab	Alternar 2D → 3D → Split
Supr / Retroceso	Eliminar la selección
Esc	Cancelar, cerrar la paleta y quitar la selección
Ctrl+Z	Deshacer
Ctrl+Shift+Z o Ctrl+Y	Rehacer
Ctrl+A	Seleccionar todo en la planta activa
Ctrl+S	Guardar ahora
Ctrl+K	Paleta de comandos
Shift al girar	Girar a saltos del incremento configurado

Aviso: los modelos generados automáticamente pueden tener errores de medidas. Verifica las medidas importantes antes de usarlas para construcción, presupuestos o trámites.

3. Guía de desarrollo

Guía para que un miembro nuevo del equipo pueda instalar el proyecto, entender cómo está organizado y hacer su primer cambio.

Contenido

- 1. Stack tecnológico
- 2. Requisitos
- 3. Instalación paso a paso
- 4. Base de datos local sin Supabase
- 5. Estructura del monorepo
- 6. Arquitectura del editor
- 7. API REST
- 8. Flujo de trabajo con Git
- 9. Pruebas, typecheck y CI
- 10. Recetas: cómo extender el editor
- 11. Convenciones de código
- 12. Solución de problemas

3.1 Stack tecnológico

Capa	Tecnología
Frontend y API	Next.js 15 (App Router), React 19, TypeScript estricto, Tailwind CSS
3D	Three.js con React Three Fiber y <code>@react-three/drei</code> (<code>OrbitControls</code>)
Estado del editor	Zustand (<code>apps/web/src/lib/editor/store.ts</code>)
Validación	Zod , con esquemas compartidos entre cliente y servidor
Base de datos	PostgreSQL en Supabase , con Prisma 6 como ORM
Pruebas	Vitest
Monorepo	pnpm workspaces
Pagos	Wompi (Colombia), con un proveedor <code>manual</code> para desarrollo
Asistente	Motor local (<code>ASSISTANT_PROVIDER=local</code>) u opcionalmente Claude (<code>ANTHROPIC_API_KEY</code>)

3.2 Requisitos

- **Node.js ≥ 20.11**. Probado con Node 22 y 24.
- **pnpm 11**. El `package.json` fija `pnpm@11.18.0`. Si no tienes pnpm, usa Corepack, que viene con Node:

```
corepack enable      # en Windows puede requerir una terminal como administrador
corepack pnpm -v     # alternativa sin permisos: anteponer "corepack" a cada comando
```

- **Git.**
- **Una base de datos PostgreSQL:** el proyecto de Supabase del equipo o una base local (ver [3.4](#)).

3.3 Instalación paso a paso

```
# 1. Clonar y entrar a la rama de trabajo
git clone https://github.com/Txllagts/ArchVision3DAI.git
cd ArchVision3DAI
git checkout Develop

# 2. Instalar dependencias
pnpm install

# 3. Variables de entorno
cp .env.example .env
# Edita .env: DATABASE_URL, DIRECT_URL y AUTH_SECRET como mínimo

# 4. Generar un AUTH_SECRET
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"

# 5. Cliente de Prisma y esquema
pnpm db:generate
pnpm db:push      # crea las tablas (ver la nota de abajo)
pnpm db:seed      # usuario demo y 3 proyectos de ejemplo

# 6. Servidor de desarrollo
pnpm dev          # http://localhost:3000
```

⚠ **Usa** `pnpm db:push`, **no** `pnpm db:migrate` **ni** `pnpm db:deploy`. Las migraciones de `packages/database/prisma/migrations` se generaron para SQLite y fallan contra PostgreSQL con el error `P3019`. Ver [Problemas conocidos](#).

⚠ `.env` está en `.gitignore`. **Nunca lo subas al repositorio** y no compartas las claves de Supabase por chat.

Variables de entorno principales

Variable	Obligatoria	Descripción
DATABASE_URL	Sí	Conexión de la aplicación. En Supabase es el Transaction Pooler (puerto 6543, con <code>?pgbouncer=true</code>)
DIRECT_URL	Sí	Conexión directa para Prisma CLI. En Supabase es el Session Pooler (puerto 5432)
AUTH_SECRET	Sí	Secreto de al menos 32 caracteres para firmar las sesiones
SESSION_TTL_DAYS	No	Duración de la sesión en días (30 por defecto)
STORAGE_DRIVER	No	<code>local</code> (por defecto, guarda en <code>./storage</code>), <code>s3</code> o <code>supabase</code>
BILLING_PROVIDER	No	<code>manual</code> (desarrollo) o <code>wompi</code>
ASSISTANT_PROVIDER	No	<code>local</code> o <code>claude</code> . Con <code>claude</code> se necesita <code>ANTHROPIC_API_KEY</code>
NEXT_PUBLIC_APP_NAME	No	Nombre comercial que muestra la interfaz

`apps/web/next.config.ts` carga el `.env` de la **raíz** del monorepo. No crees un `.env` dentro de `apps/web` .

3.4 Base de datos local sin Supabase

Para trabajar sin tocar la base compartida del equipo, hay dos opciones.

Opción A: Docker. El repositorio incluye `docker-compose.yml` con PostgreSQL 16, Redis y MinIO:

```
docker compose up -d postgres
```

```
DATABASE_URL="postgresql://archvision:archvision@localhost:5432/archvision"
DIRECT_URL="postgresql://archvision:archvision@localhost:5432/archvision"
```

Opción B: sin Docker. El paquete npm `embedded-postgres` descarga un PostgreSQL portátil. Úsalo en una carpeta **fuera del repositorio**:

```
mkdir ~/pg-local && cd ~/pg-local
npm init -y && npm i embedded-postgres
```

```
// start.mjs
import EmbeddedPostgres from "embedded-postgres";
import fs from "node:fs";

const pg = new EmbeddedPostgres({
  databaseDir: "./data",
  user: "postgres",
  password: "postgres",
  port: 54329,
  persistent: true,
});
if (!fs.existsSync("./data")) await pg.initialise();
await pg.start();
try { await pg.createDatabase("archvision"); } catch {}
console.log("PostgreSQL listo en el puerto 54329");
```

```
DATABASE_URL="postgresql://postgres:postgres@localhost:54329/archvision"
DIRECT_URL="postgresql://postgres:postgres@localhost:54329/archvision"
```

Después ejecuta `pnpm db:push` y `pnpm db:seed`. Las capturas de la [Guía de uso](#) se tomaron con esta opción.

3.5 Estructura del monorepo

```

ArchVision3DAI/
├─ apps/
│   └─ web/                                Next.js: landing, dashboard, API REST y editor
│       └─ src/
│           └─ app/
│               ├── (auth)/                /login, /register
│               ├── (app)/                  /dashboard, /projects, /settings, /trash
│               ├── (editor)/              /projects/[id]/editor
│               └─ api/                    Rutas REST (ver 3.7)
│           ├── components/
│               ├── editor/                Visores 2D/3D, barras, paneles, inspector
│               ├── projects/              Tarjetas, formularios, versiones
│               ├── billing/               Planes y facturación
│               └─ ui/                    Botones, campos y superficies base
│       └─ lib/
│           ├── editor/                   store.ts (Zustand), atajos, autoguardado
│           ├── auth/                     Sesiones, contraseñas, guards
│           ├── projects/                  Servicios de proyecto, escena y archivos
│           ├── storage/                   Almacenamiento de archivos y detección de tipo
│           ├── billing/                   Proveedores de pago
│           └─ assistant/                  Conexión del asistente con Claude
└─ packages/
    ├── types/                            Contratos: entidades, SceneDocument, comandos
    ├── validation/                        Esquemas Zod (cliente y servidor)
    ├── shared/                            Reducer de comandos, unidades, habitaciones, bounds, casa demo
    ├── geometry/                          Geometría pura: paredes con vanos, losas, cubiertas, escaleras
    ├── three-engine/                      Materiales PBR, texturas procedurales, snapping, encuadres
    ├── vision/                            Detección de muros en planos (Otsu, Hough)
    ├── assistant/                         Intenciones, planificador, diagnósticos, tutorial
    ├── billing/                           Periodos, derechos por plan, firmas de webhooks
    ├── config/                            Marca, límites y precios
    └─ database/                           Esquema Prisma, cliente y seed

```

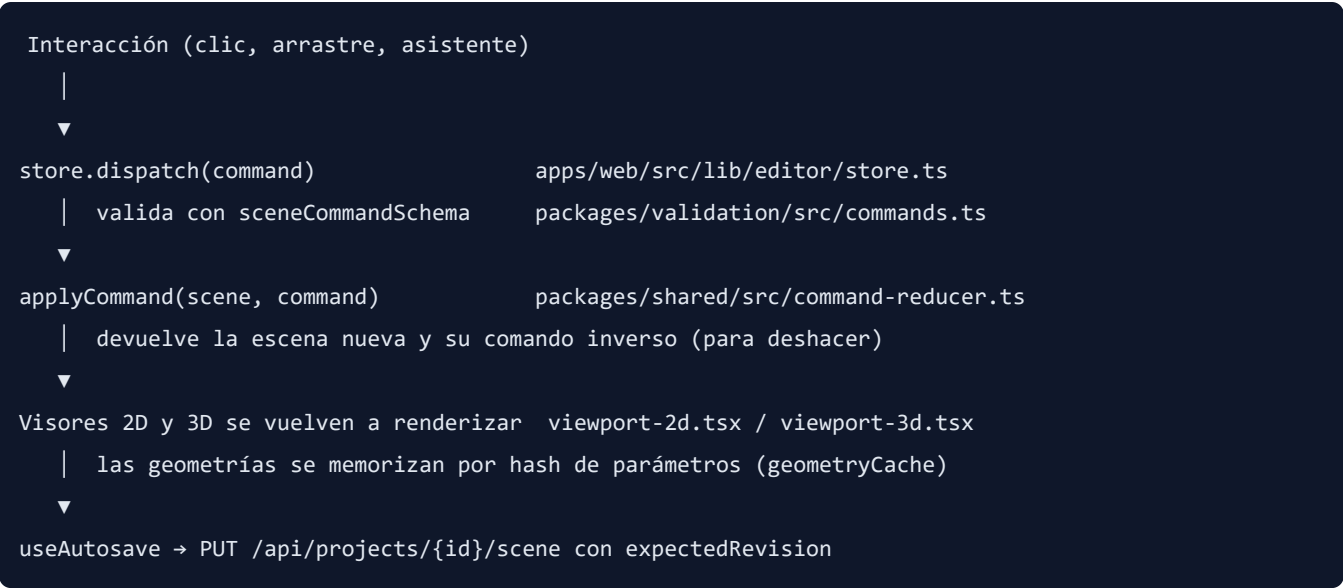
Regla de dependencias: los paquetes de `packages/` **no importan nada de** `apps/web`. `types` no depende de nadie. `geometry` y `shared` son funciones puras sin React. Solo `apps/web` conoce React y Next.

Los paquetes se publican como TypeScript sin compilar. Next los transpila gracias a `transpilePackages` en `next.config.ts`. **Si creas un paquete nuevo, agrégalo a esa lista.**

3.6 Arquitectura del editor

El modelo es un único documento JSON, `SceneDocument` (`packages/types/src/scene.ts`), con listas de `floors`, `walls`, `doors`, `windows`, `rooms`, `roofs`, `slabs`, `stairs`, `columns`, `furniture` y `materials`.

Toda modificación pasa por un comando. Ningún componente modifica la escena directamente.



Estado de la interfaz frente a estado del modelo:

Vive en SceneDocument (se guarda)	Vive solo en el store (no se guarda)
Entidades, posiciones, materiales, visibilidad y bloqueo	Herramienta activa (ToolId), selección, hover, paneles abiertos, modo de vista, cámara

Archivos clave del editor

Archivo	Responsabilidad
lib/editor/store.ts	Estado global: escena, historial de deshacer/rehacer, herramienta, selección y paneles
lib/editor/use-shortcuts.ts	Atajos de teclado
lib/editor/use-autosave.ts	Guardado con debounce y control de revisión
components/editor/editor-shell.tsx	Composición del editor
components/editor/toolbar.tsx	Lista TTOOLS de la barra izquierda
components/editor/viewport-2d.tsx	Planta en SVG: dibujo, snapping, marquee y pan
components/editor/viewport-3d.tsx	Canvas R3F: OrbitControls , plano de trabajo, arrastre, giro y marquee por frustum
components/editor/scene-objects.tsx	Una malla por tipo de entidad (WallObject , RoomFloorObject ...) con picking
components/editor/inspector.tsx	Propiedades y giro por incrementos
components/editor/command-palette.tsx	Acciones de Ctrl+K
components/editor/status-bar.tsx	TOOL_HINTS : la ayuda de cada herramienta

Más detalle en EDITOR.md y SELECTION_AND_TRANSFORMS.md .

3.7 API REST

--

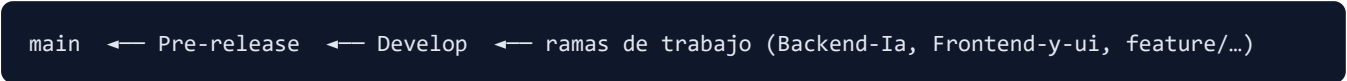
Todas las rutas están en `apps/web/src/app/api/`. Las que tocan proyectos exigen sesión (`lib/api/auth-guard.ts`) y validan el cuerpo con Zod.

Método y ruta	Uso
POST <code>/api/auth/register</code> · <code>login</code> · <code>logout</code> , GET <code>/api/auth/me</code>	Autenticación con cookie de sesión
GET, POST <code>/api/projects</code>	Listar y crear proyectos
GET, PATCH, DELETE <code>/api/projects/{id}</code>	Leer, editar y enviar a la papelera
POST <code>/api/projects/{id}/restore</code> · <code>duplicate</code>	Restaurar y duplicar
GET, PUT <code>/api/projects/{id}/scene</code>	Leer y guardar el <code>SceneDocument</code> (con <code>expectedRevision</code>)
GET, POST <code>/api/projects/{id}/versions</code>	Historial de versiones
GET, POST <code>/api/projects/{id}/files</code> y <code>/files/{fileId}</code>	Archivos del proyecto, como los planos
POST <code>/api/projects/{id}/assistant</code>	Asistente
<code>/api/billing/checkout</code> · <code>subscription</code> · <code>renewals</code> · <code>webhook/{provider}</code>	Facturación
GET <code>/api/health</code>	Comprobación de estado

Referencia completa: [API.md](#) .

3.8 Flujo de trabajo con Git

Ramas



Rama	Uso
<code>main</code>	Estable. Solo recibe merges por <i>pull request</i> desde <code>Pre-release</code>
<code>Pre-release</code>	Integración antes de pasar a <code>main</code>
<code>Develop</code>	Rama de desarrollo común
<code>Backend-Ia</code> , <code>Frontend-y-ui</code>	Ramas por área

Pasos para hacer un cambio

```
git checkout Develop && git pull
git checkout -b feature/mi-cambio
# ... trabajar ...
pnpm typecheck && pnpm test
git add <archivos>
git commit -m "feat(editor): descripción corta en imperativo"
git push -u origin feature/mi-cambio
# Abrir un pull request hacia Develop en GitHub
```

Mensajes de commit

Usa [Conventional Commits](#): `tipo(ámbito): descripción`.

Tipo	Cuándo
feat	Funcionalidad nueva
fix	Corrección de un error
docs	Solo documentación
refactor	Cambio interno sin cambiar el comportamiento
test	Pruebas
chore / ci	Configuración, dependencias, CI

Ejemplos del repositorio: `fix(storage): handle potential undefined index in readUInt32BE`, `fix(ci): add missing storage modules and fix .gitignore`.

Recomendación: un commit por cambio lógico. Por ejemplo, `a395906` mezcla tres cambios (herramienta de vista, suelos y eliminación de la página *Equipo*). Separados, sería más fácil revisarlos y revertirlos.

3.9 Pruebas, typecheck y CI

```
pnpm typecheck      # TypeScript estricto en todos los paquetes
pnpm test           # Vitest en todos los paquetes
pnpm --filter @archvision/shared test # un solo paquete
pnpm build          # compilación de producción
```

Estado en `Develop` (`a395906`) el 27/09/2026:

Paquete	Pruebas
billing	✓ 35
validation	✓ 10
vision	✓ 10
shared	✓ 40
assistant	✓ 23
geometry	✓ 13
three-engine	✗ 18 de 19. Ver Problemas conocidos
typecheck	✓ Sin errores

CI (`.github/workflows/ci.yml`) se ejecuta en cada *push* a `main` y en cada *pull request*. Instala dependencias, genera Prisma y ejecuta `typecheck`, `test` y `build`. **Un PR con pruebas en rojo no debe mezclarse.**

Las pruebas viven junto al código (`*.test.ts`) y cubren la lógica pura: el reducir de comandos, la geometría, los bounds, la visión y la facturación. Los componentes React no tienen pruebas automáticas. Si cambias un visor, pruébalo a mano en el navegador.

3.10 Recetas: cómo extender el editor

Añadir una herramienta a la barra

El commit `a395906` sirve de ejemplo: agregó la herramienta `pan`.

1. `lib/editor/store.ts` : añade el identificador al tipo `ToolId`.
2. `components/editor/toolbar.tsx` : añade una entrada a `TOOLS` con `id`, `label`, `shortcut` y un icono de `lucide-react`.
3. `lib/editor/use-shortcuts.ts` : añade un `case` con la tecla dentro del `switch`. Comprueba que la tecla no esté ya ocupada.
4. `components/editor/command-palette.tsx` : añade la acción a la paleta.
5. `components/editor/status-bar.tsx` : añade la ayuda en `TOOL_HINTS`.
6. **Visores:** en `viewport-2d.tsx` y `viewport-3d.tsx`, maneja `tool === "mi-herramienta"` en `onPointerDown`. En 3D, ajusta también el cursor y los `mouseButtons` de `OrbitControls` si hace falta.

Añadir un tipo de entidad seleccionable

Sigue los 7 pasos de `SELECTION_AND_TRANSFORMS.md` → *Agregar una clase seleccionable*. En resumen:

1. Tipo en `packages/types/src/entities.ts`.
2. Esquema en `packages/validation`.
3. Bounds en `packages/shared/src/selectable-bounds.ts`.
4. Comando en `command-reducer.ts`.
5. Geometría en `packages/geometry`.
6. Malla en `scene-objects.tsx`.
7. Pruebas.

Añadir un comando

1. Tipo en `packages/types/src/commands.ts`.
2. Esquema Zod en `packages/validation/src/commands.ts`.
3. Caso en `applyCommand` (`packages/shared/src/command-reducer.ts`). Debe devolver la escena nueva y el **comando inverso**, que permite deshacer.
4. Prueba en `packages/shared/src/editor.test.ts`.

Añadir un material al catálogo

Edita `packages/shared/src/material-catalog.ts`. Usa un `pattern` existente de `three-engine/src/patterns.ts` o crea uno nuevo. Los patrones nuevos deben cerrar sin costura; `textures.test.ts` lo comprueba.

Cambiar el esquema de base de datos

1. Edita `packages/database/prisma/schema.prisma`.
2. Ejecuta `pnpm db:generate` y `pnpm db:push` contra tu base **local**.
3. Coordina con el equipo antes de aplicarlo en Supabase.

3.11 Convenciones de código

- **Idioma:** la interfaz, los comentarios y la documentación están en español. Los identificadores de código están en inglés (`createWallGeometry` , `RoomFloorObject`).
- **TypeScript estricto.** No uses `any` . Los contratos compartidos van en `packages/types` .
- **Inmutabilidad:** el reducer nunca muta la escena; siempre devuelve objetos nuevos.
- **Unidades:** todo se expresa en **metros y radianes** internamente. La conversión a otras unidades solo ocurre en la interfaz (`packages/shared/src/units.ts`).
- **Coordenadas:** en planta, `Vector2 { x, y }` . En Three.js, la `y` de planta es el eje `z` , y la `y` de Three.js es la altura.
- **Geometría pura:** las funciones de `packages/geometry` no dependen de React ni del store, y sus resultados se memorizan en `geometryCache` con una clave que incluye todos los parámetros.
- **Funcionalidades pendientes:** se muestran deshabilitadas con la fase en la que llegan, nunca como enlaces rotos.

3.12 Solución de problemas

Síntoma	Causa y solución
'pnpm' is not recognized	pnpm no está instalado. Ejecuta <code>corepack enable</code> (como administrador) o usa <code>npm i -g pnpm@11</code>
Error: P3019 ... migration_lock.toml, sqlite	Las migraciones son de SQLite. Usa <code>pnpm db:push</code>
P2021 ... table public.User does not exist al hacer seed	El esquema no está aplicado. Ejecuta <code>pnpm db:push</code> antes de <code>pnpm db:seed</code>
@prisma/client did not initialize yet	Ejecuta <code>pnpm db:generate</code>
El editor abre con la escena vacía o no guarda	Revisa <code>DATABASE_URL</code> y la consola del servidor. El guardado usa <code>expectedRevision</code> : si otra pestaña guardó antes, recarga la página
El visor 3D va lento	La aceleración por hardware está desactivada en el navegador. La barra de estado muestra los FPS
Variables de entorno ignoradas	El <code>.env</code> debe estar en la raíz del monorepo, no en <code>apps/web</code>

4. Problemas conocidos

Problemas encontrados el 27/09/2026 al instalar la rama `Develop` (`a395906`) desde cero, ejecutar las pruebas y recorrer la aplicación. Cada entrada indica cómo reproducir el problema y una propuesta de solución.

#	Problema	Gravedad	Área
1	Las migraciones de Prisma son de SQLite y el esquema es PostgreSQL	Alta	Base de datos
2	Una prueba de <code>three-engine</code> falla y CI queda en rojo	Alta	Pruebas / CI
3	CI usa <code>DATABASE_URL</code> de SQLite	Media	CI
4	La habitación <i>Sala</i> de la casa demo muestra <code>0.00 m²</code>	Media	Editor
5	Los atajos de la barra de herramientas no coinciden con el teclado	Baja	Editor
6	<code>EDITOR.md</code> documenta atajos que no existen	Baja	Documentación

1. Migraciones de Prisma en SQLite

Reproducir:

```
pnpm db:deploy
```

```
Error: P3019
The datasource provider `postgresql` specified in your schema does not match the one
specified in the migration_lock.toml, `sqlite`.
```

Causa: el commit `42fc0c9` cambió `schema.prisma` a `provider = "postgresql"`. Las carpetas `prisma/migrations/0_init` y `20260813153636_billing`, y `migration_lock.toml`, siguen siendo de SQLite.

Solución temporal: usa `pnpm db:push`.

Solución definitiva:

1. Borra `packages/database/prisma/migrations/`.
2. Ejecuta `pnpm db:migrate --name init` contra una base PostgreSQL vacía.
3. Si la base de Supabase ya tiene tablas, márcala como aplicada: `prisma migrate resolve --applied <nombre>`.
4. Haz commit de las migraciones nuevas.

2. Prueba fallida en `three-engine`

Reproducir:

```
pnpm --filter @archvision/three-engine test
```

```
FAIL src/engine.test.ts > encuadres > calcula la caja envolvente de la casa demo
AssertionError: expected 11.2 to be close to 10
```

Causa: en `45beb28`, `computeSceneBounds` (`packages/three-engine/src/views.ts`) pasó a usar `getSelectableBounds("roof", ...)` . Esa función incluye el **alero** de la cubierta (`overhang` , 0,6 m por lado). La casa mide 10 m, así que el ancho total es 11,2 m. La prueba sigue esperando 10.

Comprobado: la prueba pasa en `0d25dc2` , el commit anterior a ese cambio.

Solución: decidir cuál es el comportamiento correcto.

- Si el encuadre debe incluir el alero, que es lo razonable para que la cámara no corte el techo, actualiza la prueba en `engine.test.ts:94-95` a `10 + 2 * overhang` .
- Si no debe incluirlo, `computeSceneBounds` debe restar el alero.

3. CI usa una `DATABASE_URL` de SQLite

`.github/workflows/ci.yml` define `DATABASE_URL: "file:./ci.db"` , y el esquema ahora es PostgreSQL.

`prisma generate` no se conecta a la base, así que hoy no falla. Pero la variable está desactualizada, y cualquier paso futuro que use la base (migraciones o pruebas de integración) fallará.

Solución: usa una URL PostgreSQL ficticia, por ejemplo `postgresql://ci:ci@localhost:5432/ci` , o un servicio `postgres` en el workflow. Agrega también `DIRECT_URL` , que ahora exige el esquema.

4. Sala muestra `0.00 m²`

Reproducir:

1. Ejecuta `pnpm db:seed` .
2. Abre **Casa Los Robles** en el editor, en vista 2D.

La *Sala* muestra `0.00 m²` . *Cocina* muestra bien sus `20.00 m²` . Se ve en la captura `img/13-vista-2d.png` .

La casa demo (`packages/shared/src/demo-house.ts`) define la Sala como un rectángulo de 6 × 5 m, así que se esperan **30 m²**. El problema parece estar en la detección de habitaciones, que se recalcula al cargar, o en cómo se asocia el polígono detectado a la habitación con nombre. Hay que revisar `packages/shared/src/rooms.ts` .

5. Atajos de la barra que no coinciden con el teclado

`toolbar.tsx` muestra `S` para **Escalera**, pero `use-shortcuts.ts` no tiene `case "s"` . Al pulsar `S` no pasa nada.

Solución: añade el `case "s": store.setTool("stair")` o quita el atajo del botón.

6. `EDITOR.md` documenta atajos que no existen

La tabla de atajos de [docs/EDITOR.md](#) incluye `W` (mover), `E` (rotar) y `R` (escalar). No están en `use-shortcuts.ts`. Tampoco incluye los atajos nuevos `H / Q` (mover vista), `Tab`, `0` ni `Ctrl+A`.

Solución: sincronizar la tabla con [la guía de uso §2.16](#), que sí refleja el código actual.